



A PROJECT PROGRESS REPORT

on

ARDUINO-BASED HEXAPOD FOR EMERGENCY AND SURVEILLANCE

(Mid-Project Status: Design, Procurement and PCB Stage)

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by (Group B9)

Md Kaif	2300970310108
Kevin Norman	2300970310096
Aryan Kumar Srivastav	2200970310048

Under the guidance of

Mr. Amanpreet Singh Saini

Assistant Professor, Department of ECE

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
GALGOTIAS COLLEGE OF ENGINEERING AND TECHNOLOGY**

Greater Noida, Uttar Pradesh

Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow

Session 2026–27 · September 2026

DECLARATION

We hereby declare that the work presented in this progress report, titled “**Arduino-Based Hexapod for Emergency and Surveillance**”, is our own work carried out under the guidance of Mr. Amanpreet Singh Saini, Assistant Professor, Department of Electronics and Communication Engineering, Galgotias College of Engineering and Technology, Greater Noida.

This is a progress (mid-project) report. It records the work completed up to the date of submission and the work that is still planned. Wherever we have used ideas, data or figures from other sources, they have been acknowledged in the text and listed in the references. The robot has not yet been assembled or tested, and no performance results are claimed in this report.

Md Kaif

2300970310108

Date: _____

Kevin Norman

2300970310096

Date: _____

Aryan Kumar Srivastav

2200970310048

Date: _____

CERTIFICATE

This is to certify that the project progress report titled “**Arduino-Based Hexapod for Emergency and Surveillance**”, submitted by **Md Kaif (2300970310108)**, **Kevin Norman (2300970310096)** and **Aryan Kumar Srivastav (2200970310048)**, is a record of the work carried out by them under my supervision in the Department of Electronics and Communication Engineering, Galgotias College of Engineering and Technology, Greater Noida.

The report describes the progress of the project up to the stage mentioned in it and is submitted for mid-project evaluation.

Mr. Amanpreet Singh Saini

Assistant Professor (Project Mentor)

Department of ECE, GCET

Name: _____

Head of Department

Department of ECE, GCET

Date: ____

ACKNOWLEDGEMENT

We would like to thank our project mentor, **Mr. Amanpreet Singh Saini**, Assistant Professor, Department of ECE, for his guidance throughout this project. His questions during our reviews, especially about how the robot would be powered and how we would prove each part works before joining it to the rest, changed the way we planned the build.

We are thankful to the Head of the Department and the faculty of the Department of Electronics and Communication Engineering for their support, and to the college for giving us access to the 3D printing facility and material, which makes the mechanical part of this project possible within a student budget.

We also thank the laboratory staff for their help, and our friends and families for their patience while this project took up most of our time.

Md Kaif

Kevin Norman

Aryan Kumar Srivastav

ABSTRACT

Search-and-rescue teams often have to inspect places that are unsafe for people — collapsed structures, rubble, or areas after a fire or gas leak. Wheeled robots are cheap and simple but get stuck on debris and steps. Legged robots handle such ground better, and a six-legged (hexapod) robot has the added advantage that it can always keep three feet on the ground, which makes it statically stable while walking.

This project aims to build a low-cost hexapod robot for emergency surveillance using an **Arduino Uno R3** as the main controller. The planned robot has six legs with three joints each, driven by **18 MG996R servos** through **two PCA9685** 16-channel PWM driver boards on a shared I²C bus. An **ESP32-CAM** on a two-axis (pan–tilt) mount, moved by two SG90 servos, is planned to stream live video over Wi-Fi and pass user commands to the Arduino over a serial link. Three **HC-SR04** ultrasonic sensors are planned for obstacle detection and an **MPU6050** IMU for measuring body tilt. Power comes from a 3S LiPo battery through a fuse and switch into four separate buck converters: two XL4016 modules for the two 6 V servo rails, one LM2596 for the 5 V logic rail and a dedicated LM2596 for the camera.

Our base paper, by Ji et al. (IEEE Access, 2024), uses reinforcement learning to improve the attitude stability of a hexapod on rough terrain and uses the static stability margin — the distance between the projection of the centre of mass and the edge of the support polygon — as its stability measure. We use its stability concepts to guide our gait and mechanical design, but we do not implement its reinforcement-learning controller, which needs far more computing power than an 8-bit microcontroller offers.

At the time of this report, the system architecture has been finalised, the components have been purchased, and a custom PCB that plugs onto the Arduino Uno as a shield (about 114 × 84 mm) has been designed in KiCad, laid out, checked and exported as Gerber files. **PCB fabrication is the next pending step.** The next stage is mechanical fabrication through 3D printing of the legs, body plate, electronics box, camera mount and trays; this stage has not started yet. Assembly, firmware, sensor and camera integration and testing are still to be done, and no performance results are reported here.

Keywords: hexapod robot, Arduino Uno, PCA9685, ESP32-CAM, custom PCB, KiCad, static stability margin, surveillance robot, 3D printing.

TABLE OF CONTENTS

Declaration.....	ii
Certificate.....	iii
Acknowledgement.....	iv
Abstract.....	v
List of Figures.....	ix
List of Tables.....	x
List of Abbreviations.....	xi
1. Introduction.....	1
1.1 Background.....	1
1.2 Project Motivation.....	1
1.3 Problem Statement.....	2
1.4 Objectives.....	2
1.5 Scope of This Progress Report.....	3
1.6 Organisation of the Report.....	3
2. Literature Review.....	4
2.1 Legged Robots for Rough Terrain.....	4
2.2 Leg Design and Kinematic Modelling.....	4
2.3 Rescue and Surveillance Robots.....	4
2.4 Summary of Reviewed Work.....	5
2.5 Research Gap and Our Position.....	6
3. Base Paper Review.....	7
3.1 Paper Details.....	7
3.2 Static Stability, Support Polygon and Stability Margin.....	7
3.3 The Hierarchical RL Control Method.....	8
3.4 Tilt-Based Gait Switching and Foot Trajectory.....	9
3.5 Results Reported in the Paper.....	9
3.6 What We Take from the Paper and What We Do Not.....	9
4. Proposed System and Architecture.....	11
4.1 Overview of the Proposed System.....	11
4.2 System Architecture.....	11
4.2.1 Why the Arduino Uno Is the Main Controller.....	12
4.2.2 How a Command Will Travel Through the System.....	13
4.3 Major Components and Their Functions.....	13
4.4 Power Architecture.....	14
4.5 Weight and Torque Budget.....	15
5. Proposed Methodology.....	17
5.1 Overall Approach.....	17
5.2 Development Phases.....	17
5.3 Planned Control Approach.....	18
5.4 Tools Used.....	18

6. Custom PCB Development.....	19
6.1 Why We Decided to Make a Custom PCB.....	19
6.2 Role of the PCB as an Arduino Uno Shield.....	19
6.3 KiCad Design Workflow.....	20
6.4 Schematic Development.....	22
6.4.1 Arduino Uno Interface.....	22
6.4.2 PCA9685 Driver Connections.....	22
6.4.3 Servo Sockets.....	23
6.4.4 Sensor Headers.....	23
6.4.5 ESP32-CAM Interface.....	24
6.4.6 Power Inputs, Zones and Support Components.....	24
6.5 PCB Layout.....	25
6.5.1 3D View of the Board.....	27
6.6 Design Verification.....	29
6.6.1 Electrical Rules Check (ERC).....	29
6.6.2 Design Rules Check (DRC).....	30
6.6.3 Other Checks.....	31
6.7 Gerber Generation.....	31
6.8 Current Status of the PCB.....	32
6.9 Checks Planned After the Board Arrives.....	32
7. Component Procurement.....	34
7.1 Procurement Approach.....	34
7.2 Components Procured.....	34
7.3 Components Deliberately Not Used.....	37
7.4 Estimated Cost.....	38
8. Mechanical Design and 3D Printing Plan.....	39
8.1 Current Stage.....	39
8.2 Parts to Be Printed.....	39
8.3 Why Dimensional Accuracy Matters.....	40
8.4 Printing and Prototyping Workflow.....	40
8.5 Mass Placement for Stability.....	41
9. Current Project Progress.....	42
9.1 Progress Summary.....	42
9.2 Work Completed So Far.....	42
9.3 Work in Progress.....	43
9.4 Work Remaining.....	43
10. Challenges and Engineering Considerations.....	44
10.1 Challenges Specific to the Current Stage.....	44
11. Expected Working of the Robot.....	46
11.1 Start-Up Sequence.....	46
11.2 Normal Operation.....	46
11.3 Battery Swap and Tethered Operation.....	47
12. Future Plan of Action.....	48

12.1 Tentative Schedule.....	48
12.2 Planned Assembly and Test Order.....	48
12.3 Testing Plan.....	49
13. Expected Final Outcome.....	50
14. Conclusion.....	51
References.....	52
Appendix A: Planned Hardware Reference for Firmware.....	53

LIST OF FIGURES

Figure 3.1	Support polygon and static stability margin during a tripod gait (drawn by us to explain the concept used in [1]).....	8
Figure 4.1	Overall system block diagram of the planned hexapod (based on the architecture in our build plan).....	12
Figure 4.2	Planned path of one user command from the phone to the leg joints.....	13
Figure 4.3	Planned power rails, their PCB input terminals and loads.....	15
Figure 4.4	Planned leg geometry and the thigh-servo torque check.....	16
Figure 5.1	Project flow from concept to testing, showing current status.....	17
Figure 6.1	KiCad schematic, sheet 1 — Arduino headers, PCA9685 modules (U2, U3), capacitors and support circuits.....	21
Figure 6.2	KiCad schematic, sheet 2 — servo sockets, sensor headers, camera connector and power terminals.....	21
Figure 6.3	Functional grouping of the shield (conceptual — the actual placement is shown in the KiCad layout figures).....	25
Figure 6.4	Completed PCB layout in KiCad (red: front copper, blue: back copper ground pour).....	26
Figure 6.5	3D view of the PCB in KiCad — top view.....	27
Figure 6.6	3D views of the PCB from both sides.....	28
Figure 6.7	Side and edge views of the PCB, showing component heights.....	28
Figure 6.8	ERC result — 0 violations, 0 errors and 0 warnings.....	29
Figure 6.9	DRC result — 0 unconnected items; the listed violations are silkscreen clearance.....	30
Figure 7.1	Some of the procured components (photograph taken by the team).....	36
Figure 7.2	(a) Arduino Uno R3 — main controller; (b) ESP32-WROOM-32 development board — programmer for the ESP32-CAM.....	37
Figure 7.3	(c) TowerPro MG996R metal-gear servo — leg actuator; (d) PCA9685 16-channel PWM driver.....	37
Figure 8.1	3D model of the hexapod.....	40
Figure 8.2	Planned 3D printing workflow — each step is checked before the next.....	40
Figure 8.3	First 3D-printed test parts.....	41

LIST OF TABLES

Table 2.1	Summary of the reviewed literature and its relevance to our project.....	5
Table 3.1	Comparison between the base paper and our project.....	9
Table 4.1	Main subsystems of the planned robot.....	11
Table 4.2	Major components and their role in the robot.....	13
Table 4.3	Estimated weight budget of the planned robot (design estimate, not measured).....	16
Table 5.1	Phase-wise methodology.....	17
Table 5.2	Software tools and their use.....	18
Table 6.1	Arduino Uno pin allocation on the shield.....	22
Table 6.2	Servo socket allocation.....	23
Table 6.3	ESP32-CAM connector (J25).....	24
Table 6.4	Power input terminals on the PCB.....	24
Table 6.5	PCB design data.....	29
Table 6.6	Fabrication files exported from KiCad.....	31
Table 6.7	PCB milestone status.....	32
Table 7.1	Component procurement status.....	34
Table 7.2	Estimated cost of purchased components (from the build plan).....	38
Table 8.1	Planned 3D-printed parts.....	39
Table 9.1	Current project progress.....	42
Table 10.1	Key engineering challenges and how the design addresses them.....	44
Table 12.1	Tentative plan of action.....	48
Table A.1	Planned I ² C devices.....	53
Table A.2	Planned servo channel map.....	53
Table A.3	Planned firmware limits.....	53

LIST OF ABBREVIATIONS

CPG	Central Pattern Generator
CoM	Centre of Mass
DOF	Degree(s) of Freedom
DRC	Design Rule Check (KiCad PCB editor)
ERC	Electrical Rule Check (KiCad schematic editor)
ESR	Equivalent Series Resistance
GND	Ground
IK	Inverse Kinematics
IMU	Inertial Measurement Unit
I²C	Inter-Integrated Circuit (two-wire serial bus)
LiPo	Lithium Polymer (battery)
MDP	Markov Decision Process
OE	Output Enable (PCA9685 pin)
PCB	Printed Circuit Board
PWM	Pulse Width Modulation
RL	Reinforcement Learning
SAC	Soft Actor-Critic (RL algorithm)
SSM	Static Stability Margin
UART	Universal Asynchronous Receiver-Transmitter

CHAPTER 1

INTRODUCTION

1.1 Background

When a building collapses or an industrial area is hit by fire or a gas leak, the first problem for a rescue team is simply knowing what is inside. Sending a person in to look is risky, and the ground is usually covered with debris, broken slabs and gaps. A small robot that can be sent in first and send back live video can reduce that risk.

Most low-cost robots built in college labs are wheeled. They are easy to build and easy to program, but a wheel needs a continuous surface. A brick or a step that is taller than about half the wheel radius is enough to stop it. Legged robots place their feet at separate points (discrete footholds), so they can step over obstacles and adjust to uneven ground. Among legged robots, the six-legged hexapod is a practical choice: it can lift three legs at a time and still stand on the other three, so it does not need to balance dynamically like a two- or four-legged robot.

Our project, **Arduino-Based Hexapod for Emergency and Surveillance**, is an attempt to build such a robot with commonly available parts, a custom PCB and 3D-printed mechanical parts, keeping the design within what a student team can build, debug and explain.

1.2 Project Motivation

The motivation for the project came from a few practical points that we discussed while choosing a final-year project:

- **Uneven ground:** wheeled robots struggle with debris, stairs and uneven surfaces that are common in disaster sites and industrial areas.
- **Operator safety:** sending people into unsafe or physically inaccessible spaces is risky; a robot can be sent in first.
- **Remote visual monitoring:** a rescue or inspection team needs live video to judge a situation from a safe distance.
- **A self-contained embedded platform:** the robot has to carry its own battery and power conversion and still control many actuators at once.
- **Cost and reproducibility:** commercial inspection and rescue robots are far too expensive for a student lab. Building from off-the-shelf parts keeps the project affordable and repeatable.

From an ECE point of view the project also covers a wide range of topics we have studied — power electronics (buck converters, current budgets, protection), embedded systems (I²C, UART, PWM), PCB design, sensors and wireless communication — in one working system.

1.3 Problem Statement

The problem we are addressing can be stated as follows:

To design and build a low-cost, six-legged walking robot, controlled by an Arduino Uno, that can move over flat and moderately uneven ground, detect nearby obstacles, and send live video to an operator over Wi-Fi from a camera that can be turned left–right and up–down.

Breaking this down, the engineering problems are:

1. **Multi-legged locomotion** with three degrees of freedom per leg (18 joints in total), so that the robot can adapt its foot positions to the ground.
2. **Controlling 18 loaded servo motors at the same time** from a small microcontroller without jitter or signal problems.
3. **Stable power distribution** in which the large, sudden servo currents do not disturb the logic circuits or the camera.
4. **Remote surveillance** through a pan–tilt camera that streams video over Wi-Fi.
5. **Reliable wiring** on a machine that vibrates every time it takes a step.

1.4 Objectives

The objectives of the project, as defined at the start, are:

1. To design and develop a low-cost six-legged (hexapod) robot using an Arduino Uno and servo motors that can walk stably over flat and moderately uneven surfaces.
2. To integrate ultrasonic distance sensing and an ESP32-CAM module so that the robot can perform basic obstacle avoidance and stream live video for surveillance.
3. To design and evaluate a control scheme that coordinates the leg movements with sensor feedback for safe navigation in emergency scenarios.

To meet these objectives we added two supporting goals of our own during the design stage: to replace the loose jumper wiring with a **custom PCB shield**, and to design a **multi-rail power system** in which servos, logic and camera each have their own supply.

1.5 Scope of This Progress Report

This is a mid-project report. It documents what has actually been done so far and what is planned next. To keep this clear, the report uses the following wording throughout:

- **Completed** — work that is finished: concept, literature study, architecture, component selection and purchase, PCB design, layout, checks and Gerber generation.
- **Pending / next** — PCB fabrication (ready to order) and 3D printing (the next stage; not started yet).
- **Planned** — everything after that: mechanical assembly, electronics integration, firmware and gait programming, sensor and camera integration, calibration and testing.

The technical descriptions of how the robot will walk, sense and stream video describe the **planned design**. The robot has not been assembled yet, so no walking, obstacle-avoidance, streaming or power-test results are reported.

1.6 Organisation of the Report

Chapter 2 reviews related literature and Chapter 3 reviews our base paper in detail. Chapter 4 describes the proposed system, its architecture and components, and Chapter 5 the methodology. Chapter 6 covers the custom PCB, which is our main completed milestone. Chapter 7 describes component procurement and Chapter 8 the mechanical design and 3D-printing plan. Chapter 9 summarises the current progress, Chapter 10 the engineering challenges, Chapter 11 the expected working of the finished robot, Chapter 12 the plan of action and Chapter 13 the expected outcome. Chapter 14 concludes the report.

CHAPTER 2

LITERATURE REVIEW

For the literature study we looked at work in four areas: hexapod mechanical design and kinematics, stability and gait control on rough terrain, rescue robots, and low-cost surveillance robots based on Arduino and ESP32 modules. This chapter summarises the papers that most directly influenced our design decisions. Our base paper [1] is reviewed separately in Chapter 3.

2.1 Legged Robots for Rough Terrain

Hexapods are widely studied because six legs give a good balance between stability and complexity. Liu et al. [3] designed the RENS H3 hexapod, inspired by the sideways movement of crabs, and used a hierarchical, model-predictive control framework with terrain-adaptive control and foot-force compensation to walk on slopes and unknown rugged ground. Their work shows that gait choice matters for stability on slopes. For our robot, though, moving sideways for long periods would put heavy load on the hip (coxa) joints, which is one reason we plan to start with the simpler forward tripod gait.

Xu et al. [4] presented a hexapod with improved knee joints and a multimodal locomotion plan. Their knee geometry raises the height of obstacles the robot can clear without adding actuators. We are using standard rotary servos at the knee, but the paper was useful in showing how leg geometry, not only software, decides what a robot can climb.

2.2 Leg Design and Kinematic Modelling

Mohamed et al. [2] designed and modelled a hexapod with telescopic legs connected to pivot joints at the hips, using analytical kinematics, inverse kinematics and geometric analysis to keep the body level on uneven ground. Their telescopic (prismatic) legs are beyond our budget, but the way they build the kinematic model and relate leg geometry to the loads on each joint is the same approach we followed when checking the torque on our MG996R thigh servos (Section 4.7).

2.3 Rescue and Surveillance Robots

Saengsint et al. [5] built an autonomous rescue hexapod with AI-based human detection and tracking. It confirms that a hexapod is a reasonable platform for rescue work, but the AI processing needs much more computing power than a small microcontroller; on-board AI adds delay on a standard embedded controller.

Kulkarni et al. [6] built an Arduino Uno based obstacle-avoiding robot that uses an ultrasonic sensor for distance measurement and an ESP32-CAM for live video that can be opened on a phone or laptop through an IP address. This is very close to the sensing and camera stack we are using. The limitation, from our point of view, is that it is a wheeled robot, which is exactly what we want to move away from.

Mahmud et al. [7] and Cruz Ulloa et al. [8] look at victim detection in robot-assisted disaster response. Mahmud et al. generate synthetic images of partly buried victims to train detection models, while Cruz Ulloa et al. compare thermal, multispectral and RGB cameras for detecting victims. Both show how important a visual link is in search-and-rescue work. They also show where our project sits: an RGB camera like the ESP32-CAM is the realistic choice for our budget, while thermal and multispectral cameras cost far more.

Jadeja et al. [9] used deep-learning object detectors on a snake robot to detect survivors under debris, and Gelfert [10] detected victims in smoke using a mobile robot with optical and low-resolution thermal cameras. Both rely on hardware that can run neural networks. For our robot the practical approach is to send the video to the operator and leave any detection to the operator's side, rather than trying to process it on the robot.

2.4 Summary of Reviewed Work

Table 2.1: Summary of the reviewed literature and its relevance to our project

Ref.	Work	Key finding	Limitation / how it affects us
[1]	Ji et al., IEEE Access, 2024 (base paper)	RL-based gait and foot-trajectory planning with the static stability margin as the stability measure; tilt-based gait switching.	Needs RL training in simulation and a GPU; not possible on an Arduino. We use the stability concepts only (Ch. 3).
[2]	Mohamed et al., Proc. IMechE Part C, 2024	Kinematic model of a hexapod with telescopic legs and body levelling.	Telescopic actuators are beyond our budget; the modelling approach helped our torque check.
[3]	Liu et al., J. Field Robotics, 2025	Lateral (crab-like) walking and hierarchical control on rugged terrain.	Long lateral walking loads the hip joints heavily; we start with a forward tripod gait.
[4]	Xu et al., J. Field Robotics, 2024	Improved knee joint raises obstacle clearance without extra actuators.	Shows the effect of leg geometry; our knee uses a standard servo.
[5]	Saengsint et al., IEEE ROBOTICS, 2023	Rescue hexapod with AI human detection and tracking.	On-board AI needs more processing than a microcontroller.
[6]	Kulkarni et al., Springer LNEE, 2024	Arduino + ultrasonic + ESP32-CAM surveillance robot with live video.	Wheeled; confirms our sensing and camera stack.
[7]	Mahmud et al., IEEE Access, 2024	Synthetic images to improve victim detection in disaster scenes.	Shows the value of a visual link; detection is outside our scope.
[8]	Cruz Ulloa et al., Applied Sciences, 2024	Compares thermal, multispectral and RGB vision for victim detection.	Thermal/multispectral cameras exceed our budget; RGB is our choice.
[9]	Jadeja et al., Scientific Reports, 2024	Deep-learning survivor detection on a snake robot.	Needs neural-network hardware; detection better done at the operator side.
[10]	Gelfert, RiTA 2022 (Springer), 2023	Real-time victim detection in smoke with a multi-sensor unit.	Specialised sensors and deep learning; beyond this project.

2.5 Research Gap and Our Position

From the literature, two kinds of work stand out. Research hexapods [1]–[5] have good stability and terrain handling, but they depend on expensive actuators, precise force sensing or heavy computation. Low-cost surveillance robots [6] are affordable and stream video, but they are almost always wheeled.

Our project sits between these two. We are not trying to match the control performance of research robots. Instead, we want to show that a hexapod built from standard hobby parts, with a proper PCB and a properly separated power system, can serve as a small surveillance platform — and that stability ideas from research, such as the static stability margin, can still guide the design of such a robot even when the controller is an 8-bit microcontroller.

CHAPTER 3

BASE PAPER REVIEW

3.1 Paper Details

Base paper: S. Ji, W. Wei, X. Liu and J. Wu, “Optimization Control of Attitude Stability for Hexapod Robots Based on Reinforcement Learning,” *IEEE Access*, vol. 12, pp. 154120–154132, 2024, doi: 10.1109/ACCESS.2024.3485506 [1].

The paper deals with a problem that is central to our project: how to keep the body of a hexapod stable (its attitude, i.e. roll and pitch) while it walks over rough ground. The authors propose a layered control method which they call **RL-FI-RW** — reinforcement learning combined with foot impedance control and an optimised reward mechanism.

3.2 Static Stability, Support Polygon and Stability Margin

The part of the paper we use most is its stability analysis. When a hexapod walks, the feet that are on the ground (the **stance** or **support** legs) form a polygon on the ground called the **support polygon**. If the vertical projection of the robot's centre of mass (CoM) falls inside this polygon, the robot is statically stable — it will not tip over even if it stops moving. If the projection moves outside the polygon, the robot falls.

To put a number on this, the paper uses the **static stability margin (SSM)** [11]: the minimum distance from the CoM projection point $PCOM$ to the edges of the support polygon:

$$dM = \min \{ d1, d2, \dots, dn \}$$

where d_i is the perpendicular distance from the CoM projection to the i -th edge and n is the number of support legs. The paper also gives expressions for the area S and centroid of the support polygon in terms of the foot coordinates. A larger margin gives a safety buffer on uneven ground; a smaller margin means the robot has to move more carefully.

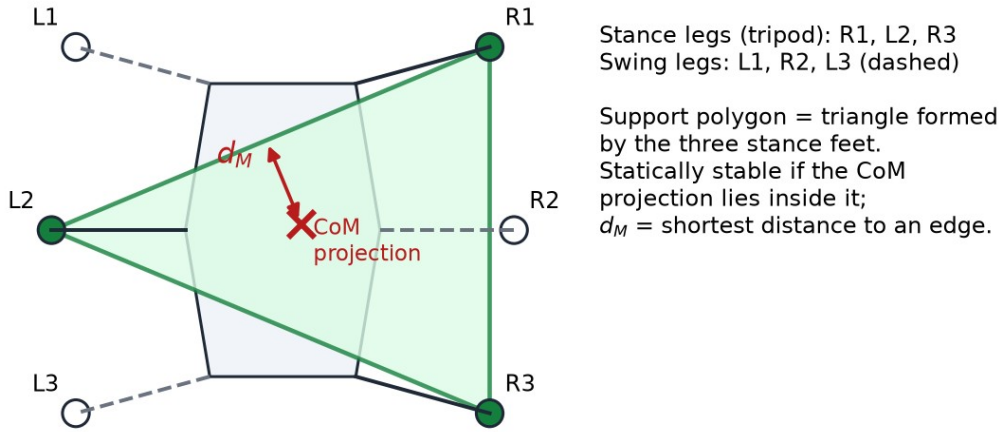


Figure 3.1: Support polygon and static stability margin during a tripod gait (drawn by us to explain the concept used in [1])

In a **tripod gait**, which is the gait we plan to use first, the legs are split into two groups of three (for example R1, L2, R3 and L1, R2, L3). One group supports the body while the other swings forward, and then they swap. The support polygon is therefore a triangle, as shown in Figure 3.1. Our mechanical plan tries to keep the CoM low and near the centre of the body so that this triangle always contains it with a reasonable margin.

3.3 The Hierarchical RL Control Method

The paper's controller has two levels:

- **Gait planner (upper level):** a gait policy network trained by reinforcement learning outputs only four values — target positions X and Y, the leg-lift height H, and the total rotation angle Ω for one gait cycle. A gait and foot-impedance coordination planner turns these into swing- and support-phase foot trajectories.
- **Joint motion controller (lower level):** an inverse-kinematics (IK) solver converts foot trajectories into joint angles, and a sliding-mode trajectory-tracking controller drives the joint motors.

The policy network is a multilayer perceptron with two hidden layers of 256 nodes. Its input is a 15-dimensional state: the body's Euler angles (3), angular velocity (3), linear velocity (3) and a six-element foot-contact vector. Foot contact is detected indirectly from joint position error and motor torque rather than from foot sensors. The authors point out that compressing the action space to four dimensions, instead of commanding every joint directly, makes training converge faster.

The reward combines posture balance, forward progress, a penalty for body rotation, a penalty for sideways drift and an energy (cost of transport) term. The balance reward is given when at least three feet are on the ground. Training uses the Soft Actor-Critic (SAC) algorithm [12].

3.4 Tilt-Based Gait Switching and Foot Trajectory

Two ideas from the paper are directly relevant to a physical robot like ours:

- **Gait switching based on body inclination.** The robot uses the angle R between its base and the ground to choose its gait: normally a three-legged (tripod) gait; a four-legged gait when R exceeds 20° ; and a five-legged gait when R reaches 25° . More legs on the ground means a larger support polygon and more margin, at the cost of speed.
- **Smooth foot trajectories.** The swing phase uses a cycloidal function, so the foot lifts off, moves forward and lands smoothly instead of jerking. The leg-lift height H is a parameter of the trajectory.

3.5 Results Reported in the Paper

All results in the paper come from **simulation** in the PyBullet physics engine, using a 15 kg hexapod model called VkHex2 with four joints per leg (24 joints in total). Training was done on a desktop computer with an Intel Core i7-7800X processor and a GTX 1080 Ti graphics card. The authors report that RL-FI-RW reduced body roll and pitch and travelled further than two CPG-based reinforcement-learning methods on randomly generated rough terrain, with height differences of up to 0.09 m. The paper itself notes that applying the method to a physical robot, with its hardware limits and sensor errors, is future work.

3.6 What We Take from the Paper and What We Do Not

It is important to be clear about this, because our project is a physical student robot and the paper is a simulation study with a much larger robot and much more computing power.

Table 3.1: Comparison between the base paper and our project

Aspect	Base paper [1]	Our project
Nature of work	Simulation (PyBullet)	Physical robot, currently being built
Robot	VkHex2, 15 kg, 4 joints per leg (24 joints)	Planned $\approx$ 2.2 kg, 3 joints per leg (18 joints)
Controller	RL policy network trained with SAC on a CPU + GPU workstation	Arduino Uno R3 (ATmega328P, 2 KB RAM)
Stability measure	Static stability margin, support polygon	Same concepts used for gait choice and mass placement
Gait	RL-optimised; tripod $\rightarrow$ 4-leg $\rightarrow$ 5-leg by tilt	Planned: tripod gait on flat ground; 4-legs-down gait on rough ground
Body tilt sensing	Base Euler angles in simulation	Planned: MPU6050 IMU on the box floor
Foot contact	Indirect, from joint error and torque	Not planned at this stage (hobby servos give no torque feedback)

Aspect	Base paper [1]	Our project
Foot trajectory	Cycloidal swing trajectory	Planned: smooth swing trajectory in the gait code
Status	Published results	No results yet; hardware stage

In short, **we do not implement the reinforcement-learning controller** and do not plan to. An RL policy of this size cannot be trained or run on an Arduino Uno. What we do use are the paper's stability ideas:

- the support polygon and static stability margin, as the reason for choosing a tripod gait and for keeping the centre of mass low and central (battery and converters under the body plate, a light camera turret over the centre);
- body inclination as a useful input for posture correction, which is why the MPU6050 is part of our design;
- the idea of moving to a gait with more legs on the ground when the terrain gets harder — our build plan already includes a four-legs-down gait for rough ground, which also reduces the load on each thigh servo;
- smooth swing trajectories rather than abrupt joint moves.

How far the IMU-based correction goes in our final firmware will depend on what the Arduino can handle alongside the gait and sensor code. This will be decided during the programming stage.

CHAPTER 4

PROPOSED SYSTEM AND ARCHITECTURE

4.1 Overview of the Proposed System

The proposed robot is a six-legged walker with three joints per leg, three ultrasonic sensors for obstacle detection, an IMU for tilt sensing, and a camera on a two-axis rotating mount that streams video over Wi-Fi. The main blocks are summarised below.

Table 4.1: Main subsystems of the planned robot

Subsystem	Planned implementation
Controller	Arduino Uno R3 — runs the gait, reads the sensors and commands every joint
Legs	6 legs $\times$ 3 joints = 18 MG996R metal-gear servos
Servo drivers	2 $\times$ PCA9685 16-channel PWM boards on one I ² C bus (addresses 0x40 and 0x41)
Camera	ESP32-CAM (AI-Thinker, OV2640) on a pan + tilt mount driven by two SG90 servos, streaming over Wi-Fi
Obstacle sensing	3 $\times$ HC-SR04 ultrasonic sensors — one in the front face of the box, one on each side face
Balance sensing	MPU6050 6-axis IMU, mounted flat on the electronics-box floor
Structure	Fully 3D printed — legs, body plate, electronics box, camera mount, battery and converter trays
Wiring	One custom PCB that plugs onto the Arduino as a shield, replacing roughly 90 loose jumper wires
Power	3S LiPo $\rightarrow$ 20 A fuse $\rightarrow$ 20 A switch $\rightarrow$ four separate buck converters

4.2 System Architecture

Figure 4.1 shows the complete planned system. Power (red) flows from the battery through the fuse and switch to a split point, then to four converters in an open tray under the body plate. Three of the converter outputs go to power zones on the PCB, and the fourth goes only to the camera. Signals (grey) run between the Arduino, the PCB, the servos and the sensors.

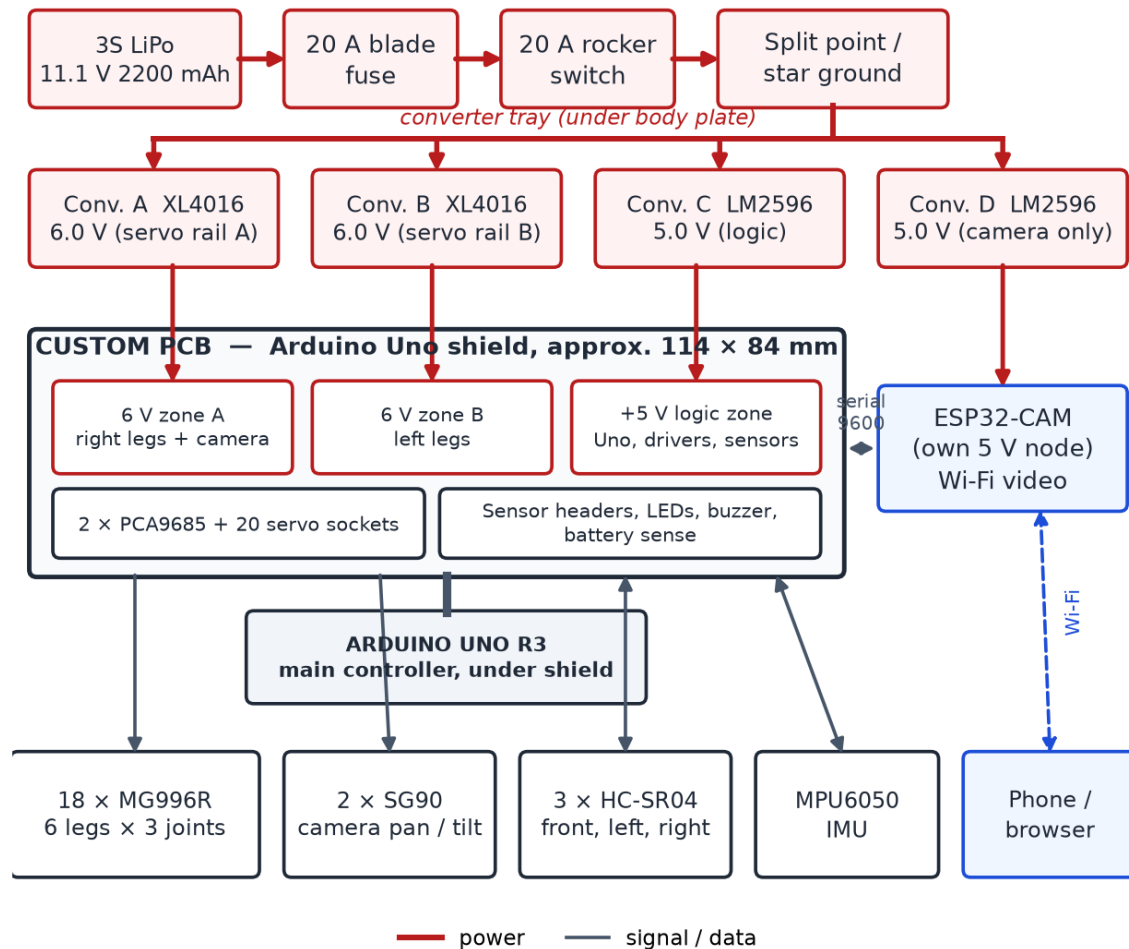


Figure 4.1: Overall system block diagram of the planned hexapod (based on the architecture in our build plan)

4.2.1 Why the Arduino Uno Is the Main Controller

A question we expect in reviews is whether the ESP32-CAM, which has a more powerful processor than the Uno, is really the brain of the robot. It is not. The Arduino Uno R3 holds the gait algorithm, reads all the sensors, decides where every joint goes and issues every command. The ESP32-CAM is a peripheral, in the same way that a Wi-Fi or GPS module is a peripheral: it contains a processor only because a camera needs one to compress video and put it on Wi-Fi. It sends the picture to the phone and passes the user's button presses to the Arduino over a serial link. It carries no control logic. The title “Arduino-based” is therefore accurate.

We also want to correct one point from our earlier presentation, where the methodology slides mentioned an Arduino Mega in two places. That was an error carried over from an older version of the plan. The controller is the **Arduino Uno R3** everywhere in the current design, and the PCB has been designed as an Uno shield.

4.2.2 How a Command Will Travel Through the System

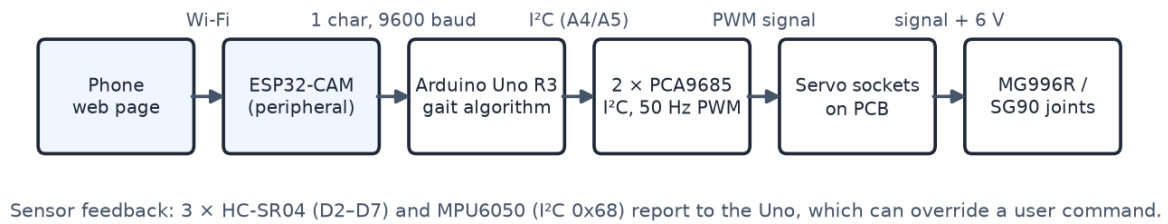


Figure 4.2: Planned path of one user command from the phone to the leg joints

1. The user presses, for example, “turn left” on the web page that the ESP32-CAM serves to the phone.
2. The camera sends a single character (L) to the Arduino over its serial line at 9600 baud. The video goes directly to the phone over Wi-Fi and never passes through the Arduino.
3. The Arduino works out where all 18 leg joints must be for the next step of the turn. This gait calculation is the only decision-making in the robot.
4. The Arduino sends these angles over I²C to the two PCA9685 boards.
5. Each PCA9685 generates the timed PWM pulses in hardware, continuously, without further work from the Arduino.
6. The pulses travel to the signal pins of the servo sockets on the PCB. The 6 V servo power comes from copper zones on the same board, not through the driver boards.
7. Each servo moves its joint to the commanded angle.
8. Meanwhile the ultrasonic sensors and the IMU report to the Arduino, which can override the user's command if it detects an obstacle or excessive tilt.

4.3 Major Components and Their Functions

Table 4.2: Major components and their role in the robot

Component	Qty	Function in the robot
Arduino Uno R3 (ATmega328P)	1	Main controller: gait, sensor reading, joint commands. The PCB plugs onto it as a shield.
MG996R metal-gear servo	18 (+1 spare)	Leg joints: hip (coxa), thigh (femur) and knee (tibia) of each leg. Stall torque about 11 kgf·cm at 6 V.
SG90 micro servo	2	Camera pan (left–right) and tilt (up–down).
PCA9685 16-channel PWM driver	2	Generate 50 Hz servo pulses in hardware; commanded over I²C. Board 1 (0x40): right legs + camera; board 2 (0x41): left legs.
ESP32-CAM (AI-Thinker, OV2640)	1	Wi-Fi video streaming and web page for user commands; serial link to the Uno.
HC-SR04 ultrasonic sensor	3	Distance to obstacles: front (“eyes”), left and right.
MPU6050 (GY-521) IMU	1	Body tilt (roll/pitch) measurement for balance and posture.
XL4016 buck converter (8–10 A)	2	6.0 V servo rails A (right legs) and B (left legs).

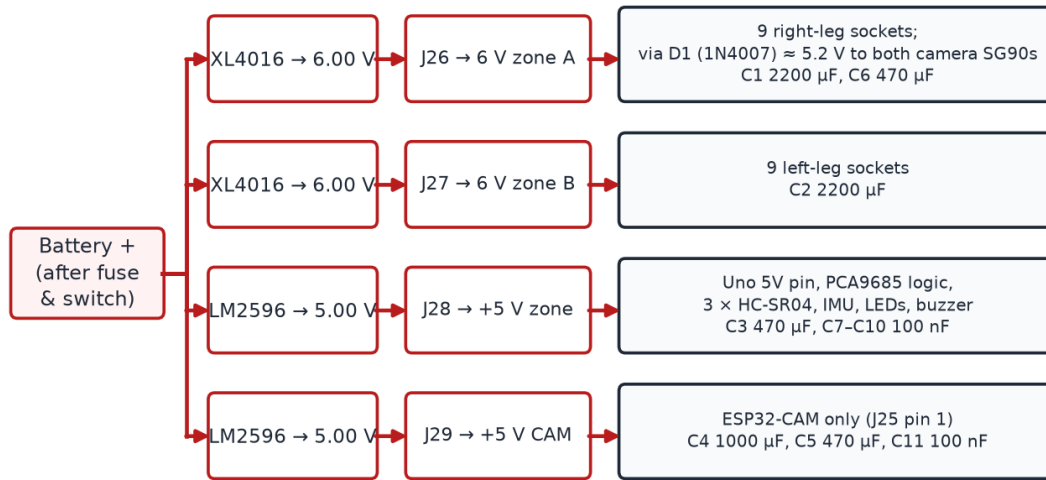
Component	Qty	Function in the robot
LM2596 buck converter	2	5.0 V logic rail and a separate 5.0 V camera rail.
3S LiPo 11.1 V 2200 mAh (XT60)	2	Power source; one fitted, one charged spare.
20 A blade fuse + holder, 20 A rocker switch	1 set	Protection and master on/off in the battery positive line.
Custom PCB (Uno shield)	1	Carries all connections, power zones, sockets and headers.

4.4 Power Architecture

We spent more time on the power system than on any other part of the design, because it is where most hobby robots fail. The main numbers from our build plan are:

- A moving MG996R draws about 0.5–0.9 A at 6 V; a stalled one about 2.5 A.
- During normal walking, with nine servos moving and nine supporting, the estimated total is about **11 A**, or about 5.5 A per servo rail. This is why each rail uses an **XL4016 rated 8–10 A** rather than a small 3 A module.
- The absolute worst case, all 18 servos stalled, is about 45 A. The fuse, switch and battery wiring must survive this long enough for the 20 A fuse to blow.
- The logic rail load (Uno, both driver boards' logic, IMU, three ultrasonic sensors, LEDs and buzzer) is estimated at about 160 mA.
- The ESP32-CAM draws about 180–250 mA normally, but bursts to about 700 mA when it transmits over Wi-Fi.

The camera's current bursts are the reason it has its own converter. If it shared a supply with the Arduino, each burst could pull the 5 V rail down and reset the Arduino in the middle of a step — a fault that would look like a software bug. Figure 4.3 shows the four rails and what each one feeds.



Battery sense: J30 → R1 100 kΩ / R2 47 kΩ divider → A0 (C12 100 nF). All negatives meet on the bottom ground pour (star point).

Figure 4.3: Planned power rails, their PCB input terminals and loads

The design follows four rules that we decided not to break:

1. **Servos never take power from the Arduino's 5 V pin.** The Uno's regulator can only supply about 0.5 A. Servo power comes only from the 6 V zones on the PCB.
2. **The battery voltage never goes to the Uno's VIN pin or barrel jack.** The 5 V pin is fed directly from converter C, bypassing the on-board regulator.
3. **One ground, one point.** All negatives meet at a single star point. If grounds join in two places, servo current can flow through the signal ground and cause random I²C failures, only when the robot moves.
4. **The camera never shares a rail with the Arduino.**

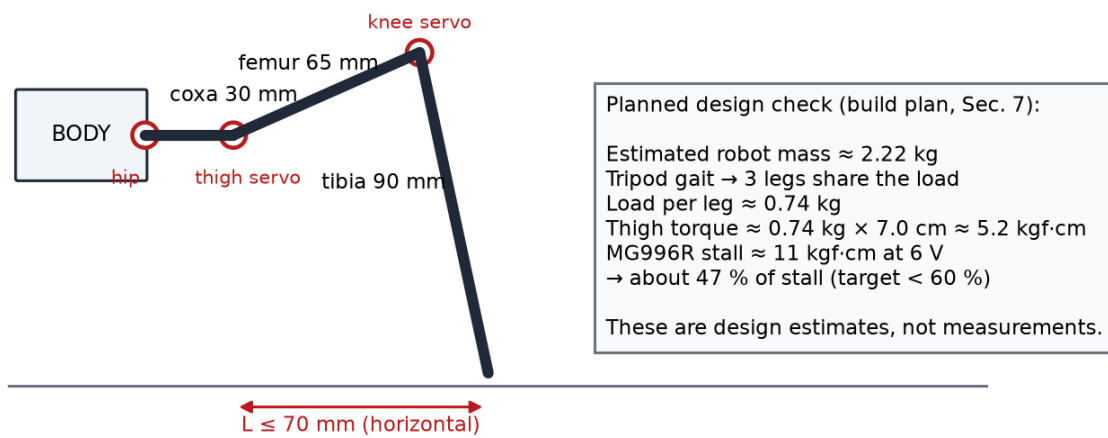
The converters are placed in an open, slotted tray under the body plate rather than inside the electronics box. They are the hottest parts of the robot, and in open air they cool without a fan. Placing them low also lowers the centre of mass. We plan to use two identical battery packs, one fitted and one charged as a spare, rather than wiring two packs together, because a second fitted pack would add about 185 g at the worst possible moment. The planned runtime estimate from the build plan is about 15 minutes of continuous walking per pack; this is an estimate and will be measured during testing.

4.5 Weight and Torque Budget

A hexapod that is too heavy does not walk badly — it does not walk at all, and there is no software fix. So before printing anything we worked out the weight and the load on the most heavily loaded joint, the thigh (femur).

Table 4.3: Estimated weight budget of the planned robot (design estimate, not measured)

Item	Estimated weight	Note
18 × MG996R servos	990 g	55 g each; fixed
Printed legs (6 sets)	340 g	About 57 g per leg
Body plate	150 g	4 mm thick with a stiffening rim
Electronics box + lid	130 g	2.4 mm walls
Camera turret (2 × SG90, mount, ESP32-CAM)	68 g	At the top — worst place for weight
3S 2200 mAh LiPo	185 g	Under the body plate
Battery tray + converter tray	60 g	Under the body plate
Arduino, PCB, converters, wiring, screws, inserts	300 g	Wiring is always heavier than expected
Total	≈ 2.22 kg	Target: stay under 2.3 kg

**Figure 4.4:** Planned leg geometry and the thigh-servo torque check

With about 2.22 kg carried on three legs in a tripod gait, each leg carries about 0.74 kg. With the foot kept within 70 mm horizontally of the thigh joint, the thigh servo torque is about $0.74 \text{ kg} \times 7.0 \text{ cm} \approx 5.2 \text{ kgf}\cdot\text{cm}$, which is about 47 % of the MG996R's stall torque at 6 V. Our target is to stay below 60 %, above which these servos run hot, drift and wear quickly. The leg segment lengths (coxa 30 mm, femur 65 mm, tibia 90 mm) were chosen with this margin in mind. With a four-legs-down gait on rough ground the load per leg drops to about 0.56 kg and the torque to about 3.9 kgf·cm (about 35 % of stall). These figures will be checked against the real, weighed robot during assembly.

CHAPTER 5

PROPOSED METHODOLOGY

5.1 Overall Approach

The central idea of our methodology is to **prove every expensive part on its own before it is connected to anything else**. A wiring mistake on a LiPo battery can deliver tens of amperes; a mis-set converter can destroy all 18 servos in seconds. So the work is ordered so that each stage is tested before the next one depends on it. Figure 5.1 shows the complete project flow and where we currently stand.

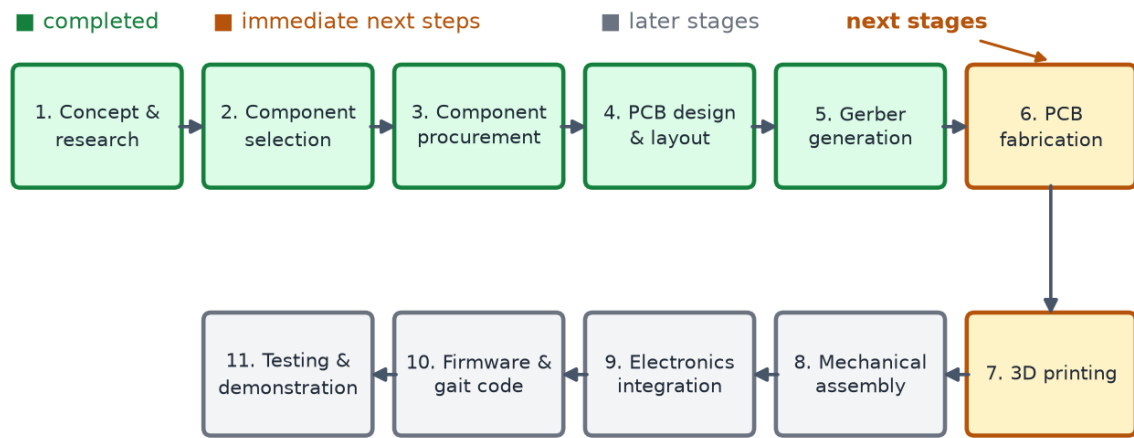


Figure 5.1: Project flow from concept to testing, showing current status

5.2 Development Phases

Table 5.1: Phase-wise methodology

Phase	Activities	Status
1. Architecture and circuit mapping	Finalise multi-rail power distribution; define every connection; design the custom PCB shield in KiCad.	Completed (PCB fabrication pending)
2. Mechanical fabrication	3D print a test piece and one leg; check servo fit; print remaining legs, body plate, box, camera mount and trays.	Next stage (not started)
3. Power and bench testing	Set and verify all four converters; build the power harness; single-servo test on the 12 V bench adapter.	Planned
4. Board bring-up and assembly	Solder and continuity-check the PCB; fit modules; I ² C scan; assemble legs one at a time.	Planned (after PCB arrives)
5. Kinematics and gait programming	Inverse kinematics for the coxa–femur–tibia legs; tripod gait with stability considerations.	Planned
6. Sensor and wireless integration	MPU6050 tilt feedback; ESP32-CAM streaming and web commands; ultrasonic obstacle checks.	Planned
7. Safety interlocks	Servo master-disable at start-up; stop forward motion when an obstacle is detected within a set distance (30 cm planned); low-battery sit-down.	Planned
8. Testing and demonstration	Electrical, mechanical, thermal and behavioural checklists; tethered and battery demos.	Planned

5.3 Planned Control Approach

The planned firmware will follow the steps below. None of this has been written for the final robot yet; the plan is to start with a single-servo test on the bench and build up from there.

- **Inverse kinematics:** convert a desired foot position (x, y, z) into the three joint angles (θ_1 , θ_2 , θ_3) of each leg using the geometry of the coxa, femur and tibia.
- **Tripod gait sequencing:** alternate two groups of three legs between swing and stance, so that three feet always form a support triangle.
- **PWM through the PCA9685s:** both drivers run at 50 Hz; a servo pulse of roughly 0.5–2.5 ms corresponds to counts of about 102–512, with centre near 307. Each servo will be calibrated individually.
- **IMU-based posture correction:** use pitch and roll from the MPU6050 to apply small corrections to the body posture on sloped ground.
- **Obstacle interlock:** poll the three HC-SR04 sensors in the main loop and override forward commands if an obstacle is too close.
- **Memory discipline:** the Uno has only 2 KB of RAM, so fixed gait tables will be stored in PROGMEM, the String class avoided and integer arithmetic preferred.

5.4 Tools Used

Table 5.2: Software tools and their use

Tool	Use in the project
KiCad	Schematic capture, PCB layout, ERC/DRC checks and Gerber export (completed)
Arduino IDE	Firmware for the Uno and the ESP32-CAM (planned)
VS Code	Code editing (planned)
Git / GitHub	Version control for design files and code
3D modelling / slicer software (college lab)	Preparing printable parts (next stage)

CHAPTER 6

CUSTOM PCB DEVELOPMENT

The custom PCB is the main completed engineering milestone of the project so far. This chapter explains why we decided to make one, how it was designed in KiCad, what it contains, how it was checked, and where it stands now.

Current PCB status

The PCB design has been completed and the Gerber fabrication files have been generated. PCB fabrication/ordering is the next pending step.

6.1 Why We Decided to Make a Custom PCB

Our first plan was to connect everything with jumper wires on a breadboard, the way most student robots are built. When we counted the connections, it came to roughly **90 loose jumper wires**: 20 three-wire servo connections, the I²C bus, six ultrasonic signal lines, the camera serial link, LEDs, buzzer and several power feeds. On a robot that shakes every time a foot lands, this is a problem for several reasons:

- **Loose contacts:** jumper wires work loose under vibration. A servo signal that drops out for a moment makes a leg twitch; an I²C line that drops out stops every servo on that board.
- **Current capacity:** breadboard strips and thin jumpers are not made to carry several amperes of servo current. They heat up and the voltage drops.
- **Debugging:** with 90 near-identical wires, finding one wrong or broken connection can take hours.
- **Wrong connections are easy to make:** for example, powering a servo from the Arduino's 5 V pin, or connecting the 3.3 V camera directly to a 5 V Arduino pin.

A PCB solves all of these at once. Connections become copper tracks and zones, power paths can be made as wide as the current needs, sockets are labelled on the silkscreen, and some wrong connections become physically impossible because the copper for them simply does not exist.

6.2 Role of the PCB as an Arduino Uno Shield

The board is designed as a **shield** that plugs directly on top of the Arduino Uno R3 through stacking headers, so there are no wires at all between the Arduino and the main board. The reference plan sizes the board at about **114 × 84 mm**. The two PCA9685 driver boards plug into

female headers on the shield, and all 20 servos, the three ultrasonic sensors, the IMU lead and the camera cable plug into connectors on it. The four converter outputs arrive on screw terminals.

The shield carries only 5 V logic and signals to the Arduino. The high servo currents stay in the 6 V zones and never pass through the Arduino or the driver modules.

6.3 KiCad Design Workflow

We used KiCad for the complete design. Our workflow was:

1. **Connection definition:** before opening KiCad, we wrote down every connection of the robot in tables — every Arduino pin, every driver channel, every sensor pin and every power terminal (summarised in Section 6.4).
2. **Schematic capture:** starting from KiCad's Arduino Uno shield template, we drew the schematic sheet by sheet: power inputs and zones, driver boards, servo sockets, sensor headers, camera interface, and the small support circuits.
3. **Electrical Rule Check (ERC):** run on the schematic to find unconnected pins, conflicting outputs and power pins without a source.
4. **Footprint assignment:** standard footprints for resistors, capacitors, headers and terminals; for the modules (PCA9685, IMU, camera), footprints were checked against the actual purchased modules, since breakout boards from different sellers do not always match.
5. **Board layout:** board outline, placement of connectors and modules, power zones, routing of signals, and the ground pour.
6. **Design Rule Check (DRC):** run on the layout to find clearance violations, unrouted connections and track-width problems.
7. **Gerber and drill file export** for fabrication.

The schematic is drawn on two sheets. The first sheet (Figure 6.1) holds the Arduino shield headers J1–J4, the two PCA9685 modules, the bulk and decoupling capacitors, the camera-servo diode D1, the battery-sense divider, the camera serial divider, the LEDs and the buzzer driver. The second sheet (Figure 6.2) holds the 20 servo sockets, the sensor headers J21–J24, the camera connector J25, the power terminals J26–J30 and the programming jumper JP1.

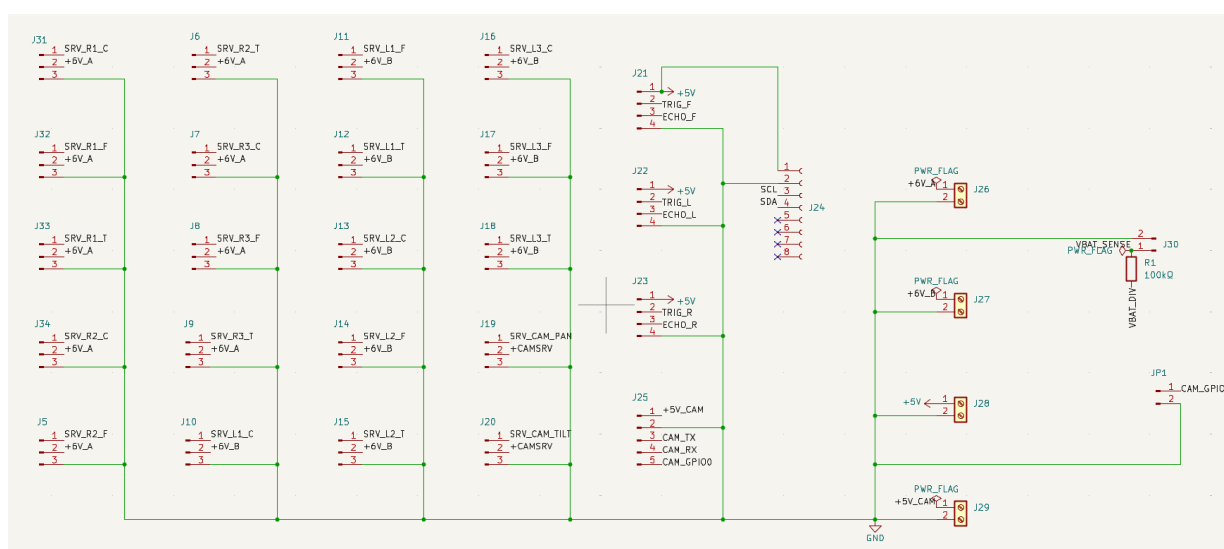
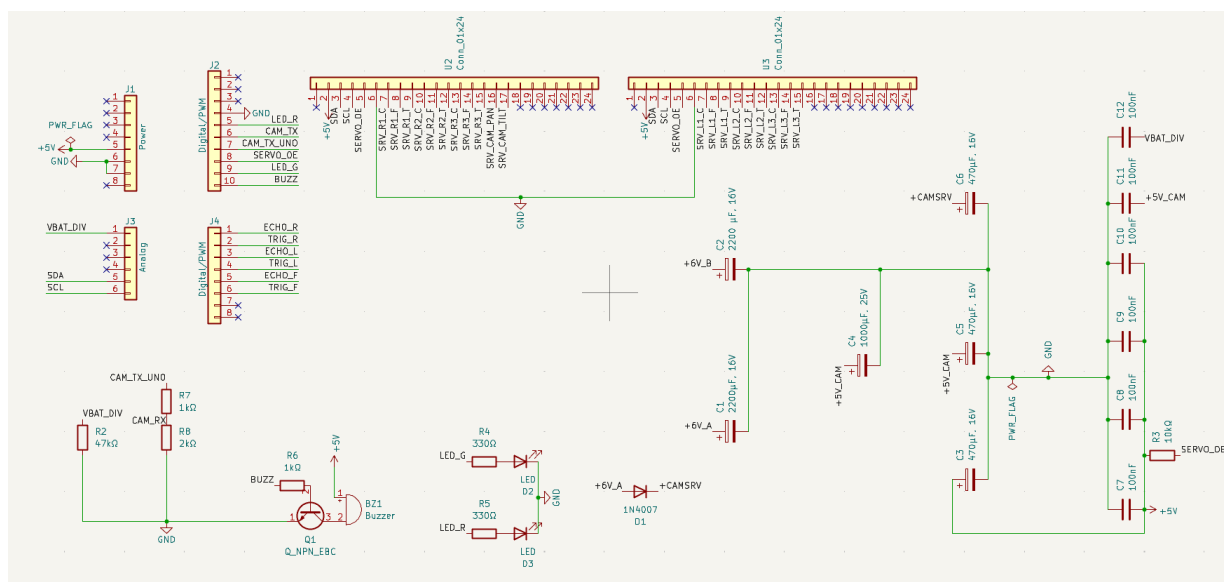


Figure 6.2: KiCad schematic, sheet 2 — servo sockets, sensor headers, camera connector and power terminals

Two details of the schematic are worth pointing out. First, the PCA9685 boards are drawn as 24-pin connector symbols (U2 and U3), because on our PCB they are plug-in modules seated in female headers, not chips soldered to the board. Second, the small blue crosses on unused pins are KiCad **no-connect flags**, placed with the Q key. They mark pins that we left unconnected on purpose — for example D0/D1, VIN and the unused driver channels — so that the checker knows these pins are not forgotten wiring.

6.4 Schematic Development

6.4.1 Arduino Uno Interface

Every Uno pin has a defined job. The I²C pins are shared by both drivers and the IMU. D0 and D1 are deliberately left unconnected because they are used for USB programming; anything connected to them would stop us uploading code. A0 is used for battery voltage sensing and A1–A3 are left free as spare pads.

Table 6.1: Arduino Uno pin allocation on the shield

Uno pin	Connected to	Purpose
5V	+5 V logic zone (from converter C)	Power in — not through VIN or the barrel jack
VIN	Not connected	Battery voltage here would overheat the Uno's regulator
A4 / A5	SDA / SCL of both PCA9685s and the IMU header (J24)	Shared I ² C bus
A0	R1/R2 divider junction, C12 (100 nF) to GND	Battery voltage sensing
A1–A3	Spare pads	Kept free for later use
D0 / D1	Not connected	Reserved for USB programming
D2 / D3	J21 — front HC-SR04 TRIG / ECHO	Front obstacle sensor
D4 / D5	J22 — left HC-SR04 TRIG / ECHO	Left obstacle sensor
D6 / D7	J23 — right HC-SR04 TRIG / ECHO	Right obstacle sensor
D8	R6 (1 k Ω) → base of Q1	Buzzer driver
D9	R4 (330 Ω) → green LED	Status LED
D10	OE of both PCA9685s, R3 (10 k Ω) pull-up to +5 V	Servo master disable
D11	R7 (1 k Ω) → camera U0R, R8 (2 k Ω) to GND	Serial to camera, 5 V → 3.3 V
D12	Camera U0T	Serial from camera
D13	R5 (330 Ω) → red LED	Low-battery / fault LED

6.4.2 PCA9685 Driver Connections

Each PCA9685 module plugs into female headers on the shield. From the 6-pin end header, GND goes to the ground pour, OE to D10, SCL and SDA to A5 and A4, and VCC to the +5 V logic zone (logic power only, a few milliamperes). The module's **V+ terminal is left unconnected**. On the output block, only the PWM row and the GND row are connected; the middle V+ row is placed on no net, so servo power can never reach the module. A socket may be fitted there for mechanical grip only.

This choice is deliberate. The PCA9685 breakout has a V+ terminal and it is tempting to feed 6 V into it and plug the servos straight into the module. But nine MG996R servos can draw about 6 A together, and the thin tracks on a small breakout board are not designed for that. In our design the driver boards carry only signals; the 6 V comes from wide copper on our own board, straight to the servo sockets.

The second board (U3) must answer to a different I²C address. On U3 only, the A0 address pads will be bridged with solder, changing its address from 0x40 to 0x41. The expected I²C scan result after assembly is 0x40, 0x41 and 0x68 (the MPU6050).

The OE (output enable) line is active low. The 10 k Ω pull-up R3 holds it high while the Uno is booting or resetting, so all 20 servo outputs stay off until the firmware has written a safe pose and deliberately pulls D10 low. This is what should stop the robot from jerking into a random pose at power-on.

6.4.3 Servo Sockets

There are 20 three-pin servo sockets in the standard servo order (pin 1 signal, pin 2 positive, pin 3 ground). Each socket is labelled on the silkscreen with its leg and joint so that during assembly we read the label rather than the reference number.

Table 6.2: Servo socket allocation

Sockets	Labels	Driver / channels	Power zone
J31, J32, J33	R1-C, R1-F, R1-T (right front hip, thigh, knee)	U2 (0x40) ch 0–2	6 V zone A
J34, J5, J6	R2-C, R2-F, R2-T (right middle)	U2 ch 3–5	6 V zone A
J7, J8, J9	R3-C, R3-F, R3-T (right rear)	U2 ch 6–8	6 V zone A
J10, J11, J12	L1-C, L1-F, L1-T (left front)	U3 (0x41) ch 0–2	6 V zone B
J13, J14, J15	L2-C, L2-F, L2-T (left middle)	U3 ch 3–5	6 V zone B
J16, J17, J18	L3-C, L3-F, L3-T (left rear)	U3 ch 6–8	6 V zone B
J19	CAM-PAN (SG90)	U2 ch 9	Zone A via D1 ($\approx$ 5.2 V)
J20	CAM-TILT (SG90)	U2 ch 10	Zone A via D1 ($\approx$ 5.2 V)

A small detail that may look odd in the schematic: the first four servo sockets are numbered J31–J34 instead of J1–J4. KiCad's Arduino shield template already uses J1–J4 for the four Arduino stacking headers, and renaming them would risk losing the verified header spacing. So those four sockets were moved to the end of the numbering. It makes no difference electrically.

Channels 11–15 on board 1 and 9–15 on board 2 are unused, leaving 12 spare servo outputs for future additions.

6.4.4 Sensor Headers

- **J21, J22, J23 (HC-SR04 front, left, right):** 4-pin headers — VCC (+5 V zone), TRIG, ECHO, GND. The sensors will be mounted on the inside of the box walls and connected with four-wire Dupont cables of about 150 mm.
- **J24 (MPU6050 lead):** VCC (+5 V; the GY-521 module has its own 3.3 V regulator), GND, SCL, SDA. The IMU is not mounted on the PCB itself: it has to lie flat and square to

measure true body tilt, so it will sit on the box floor on foam tape with its X arrow pointing to the front of the robot.

6.4.5 ESP32-CAM Interface

The camera connects through a single five-wire cable at J25:

Table 6.3: ESP32-CAM connector (J25)

J25 pin	Camera pin	Connected to
1	5V	+5 V CAM node, fed only from terminal J29 (converter D); C4 1000 μ F, C5 470 μ F and C11 100 nF across it
2	GND	Ground pour
3	U0T (camera transmit)	Uno D12 directly — a 3.3 V signal reads as HIGH on a 5 V Uno
4	U0R (camera receive)	Junction of R7 (1 k Ω from D11) and R8 (2 k Ω to GND), dividing 5 V down to about 3.3 V
5	GPIO0	Jumper JP1: shorted to GND = programming mode; open = normal running

The ESP32-CAM has no USB socket. We plan to program it using the ESP32-WROOM-32 development board that we already have, used as a USB-to-serial adapter (its EN pin tied to GND so that its own processor stays idle). JP1 on our board puts the camera into programming mode without removing any wires.

6.4.6 Power Inputs, Zones and Support Components

Table 6.4: Power input terminals on the PCB

Terminal	Source	Feeds
J26	Converter A, 6.00 V	6 V zone A $\rightarrow$ pin 2 of the nine right-leg sockets; through D1 to J19 and J20. C1 (2200 μ F) across the zone; C6 (470 μ F) after the diode.
J27	Converter B, 6.00 V	6 V zone B $\rightarrow$ pin 2 of J10–J18. C2 (2200 μ F) across the zone.
J28	Converter C, 5.00 V	+5 V zone $\rightarrow$ Uno 5V pin, U2/U3 VCC, J21–J24 VCC, LEDs, buzzer. C3 (470 μ F).
J29	Converter D, 5.00 V	+5 V CAM node $\rightarrow$ J25 pin 1 only; touches nothing else except ground.
J30	Battery positive after fuse and switch (thin wire)	R1 (100 k Ω) $\rightarrow$ A0 $\rightarrow$ R2 (47 k Ω) $\rightarrow$ GND, with C12. Divides 12.6 V down to about 4.03 V.

Some of the small parts deserve a word of explanation because each one solves a specific problem:

- **C1, C2 (2200 μ F):** one per 6 V servo zone, right beside the sockets, to absorb the current spike when nine servos start moving in the same instant.
- **C4 + C5 (1000 μ F + 470 μ F) on the camera node:** hold the camera's voltage up through its Wi-Fi transmit bursts. C4 is a 25 V part we already had; a higher voltage rating than needed is fine.

- **C7–C11 (100 nF):** one beside each module's power pin to suppress high-frequency noise.
- **D1 (1N4007):** the SG90 camera servos are rated 4.8–6.0 V, so 6.00 V is at their limit. The diode drops about 0.8 V, giving them about 5.2 V while the MG996Rs keep their full 6 V.
- **R7/R8 divider:** protects the 3.3 V camera input from the Uno's 5 V output.
- **Q1 (BC547) and BZ1:** the buzzer draws more current than an Arduino pin should supply, so a transistor switches it.
- **R1/R2 + C12:** battery sensing. The firmware formula will be $V_{BAT} = \text{analogRead}(A0) \times 0.01528$ (i.e. $5.0/1023 \times 147/47$). The planned thresholds are a warning at 10.5 V and a safe sit-down with servos disabled at 9.9 V.

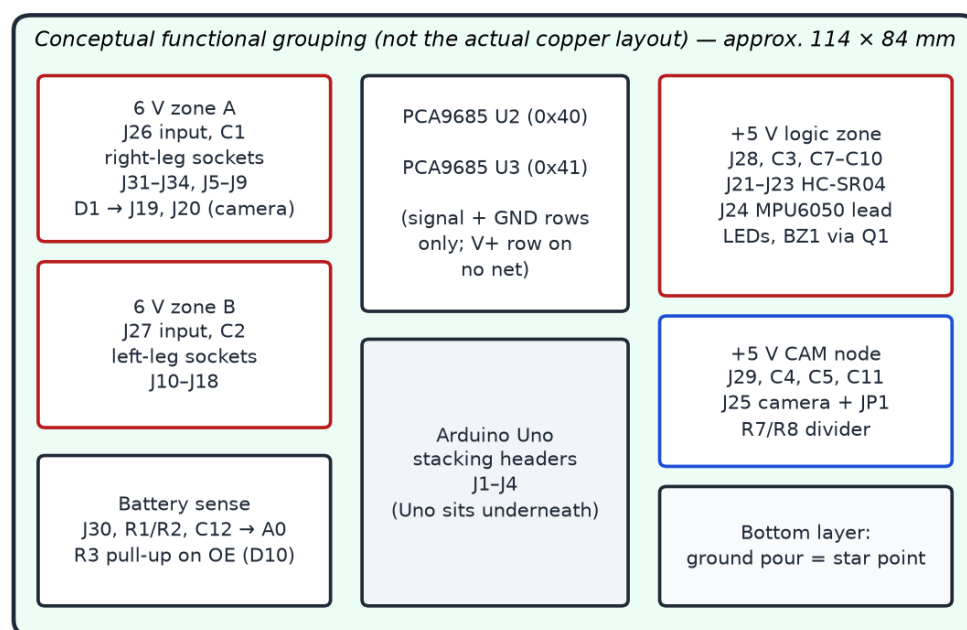


Figure 6.3: Functional grouping of the shield (conceptual — the actual placement is shown in the KiCad layout figures)

6.5 PCB Layout

With the schematic complete, the layout was done in KiCad's PCB editor. The main considerations were:

- **Board outline and shield headers:** the Uno header positions come from the KiCad shield template, so the board lines up with the Arduino underneath. The 3V3, AREF, IOREF and RESET pins pass through unused, but the header pins are kept so that the board is mechanically a proper shield.
- **Separate power zones:** 6 V zone A, 6 V zone B, the +5 V logic zone and the +5 V CAM node are separate copper areas. The two 6 V zones are not connected to each other, and the

camera node is not connected to the logic node. Servo power reaches the sockets through these copper areas rather than through thin tracks.

- **Ground:** every negative lands on a single ground pour on the bottom copper layer. On the board, this pour acts as the star point.
- **Connector arrangement:** the right-leg sockets are grouped on the zone A side and the left-leg sockets on the zone B side, so that each leg's three cables go to one area of the board. Screw terminals for the converter inputs are placed at the board edge, and the sensor headers, camera connector and JP1 are grouped so that cables can be routed neatly inside the box.
- **Bulk capacitors close to the loads:** C1 and C2 sit right beside the servo sockets they serve, and the camera capacitors sit at the camera connector.
- **Silkscreen labels:** every servo socket carries its joint label (R1-C, R1-F, R1-T and so on), and terminals are marked with polarity and voltage.
- **PCA9685 footprint:** the channels on the driver modules sit in four groups of four with uneven gaps, so the footprint was made to match the real module rather than assuming an even 2.54 mm pitch across the block.

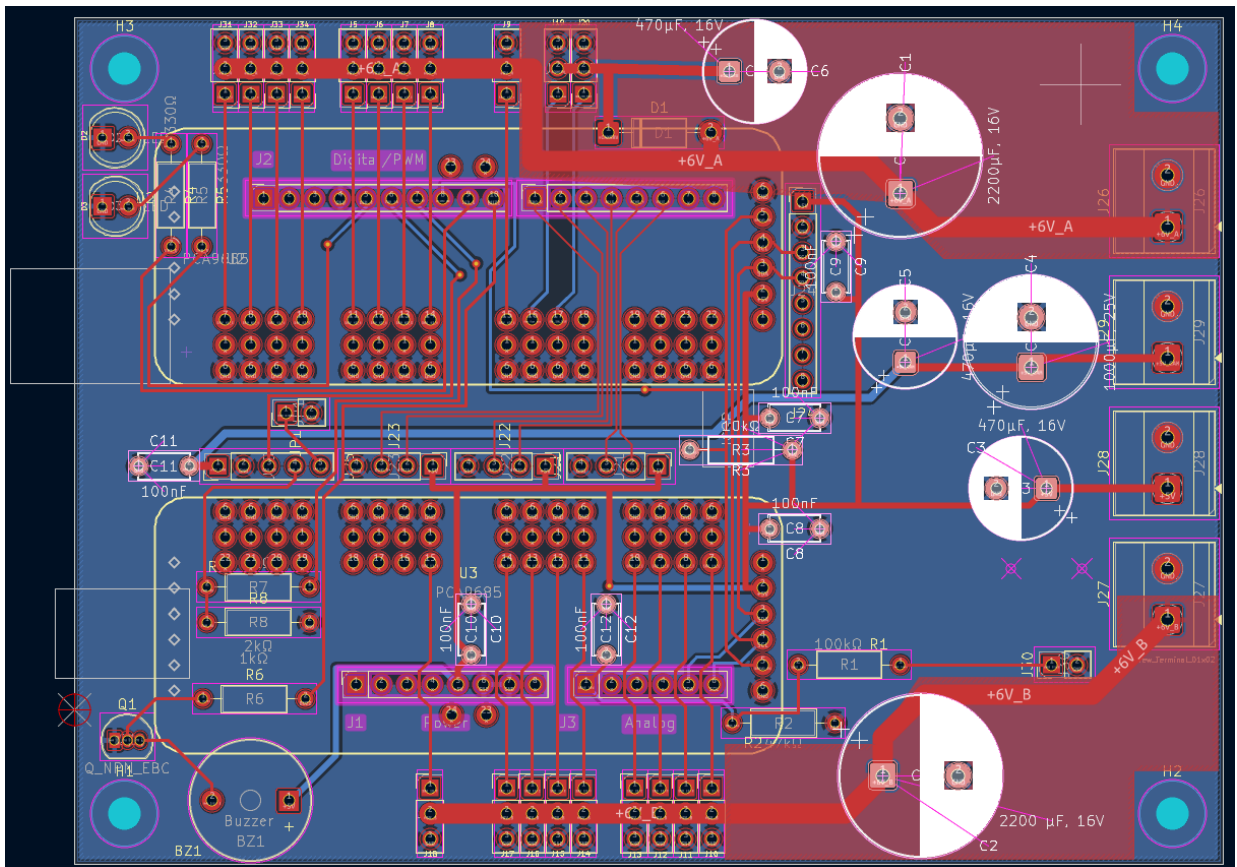


Figure 6.4: Completed PCB layout in KiCad (red: front copper, blue: back copper ground pour)

In the finished layout (Figure 6.4), the right-side leg sockets and the two camera-servo sockets run along the top edge and the left-side leg sockets along the bottom edge. The four converter

terminals J26–J29 sit on the right edge. The wide +6V_A and +6V_B copper areas run from terminals J26 and J27 to their socket rows, with the 2200 μ F capacitors C1 and C2 placed on these areas. The two PCA9685 module footprints occupy the middle of the board, and the four corner holes H1–H4 are for mounting. The blue area is the ground pour on the back copper layer.

6.5.1 3D View of the Board

KiCad's 3D viewer was used to check the assembled board before exporting the fabrication files. It shows every through-hole part at its real size, which helps catch problems that a flat layout hides — parts that are too close together, connectors facing the wrong way, or components that are too tall for the space they will sit in.

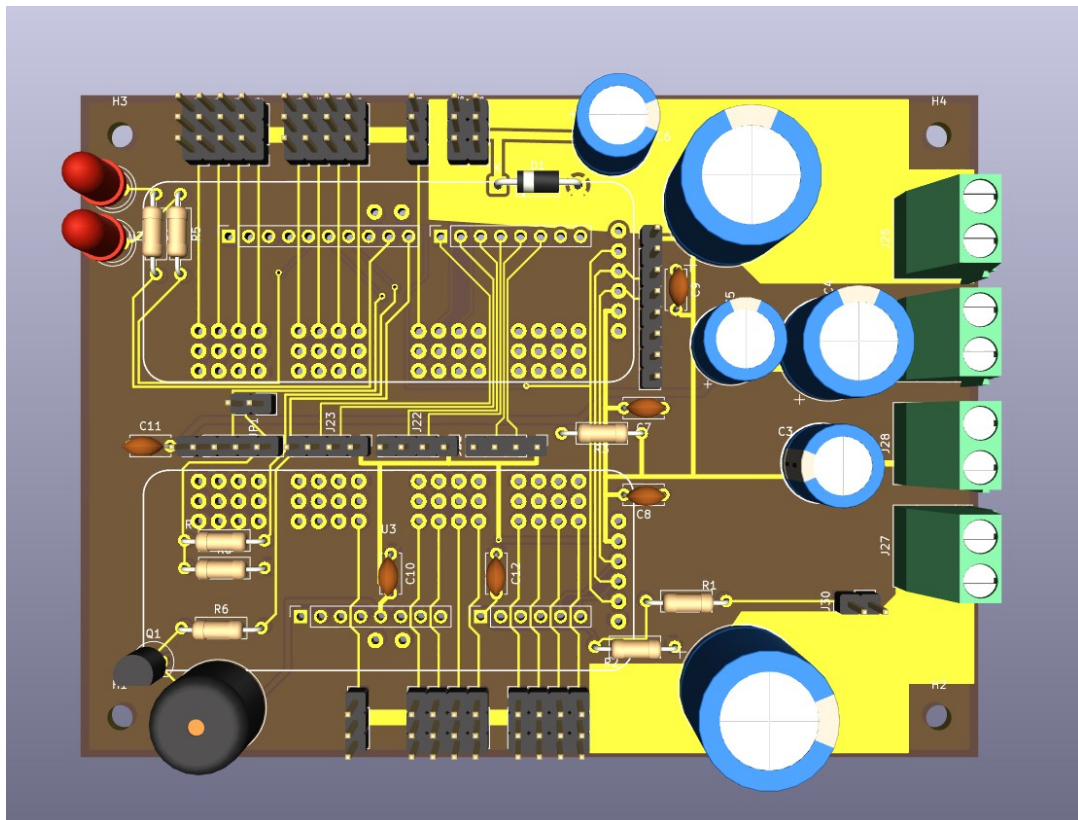


Figure 6.5: 3D view of the PCB in KiCad — top view

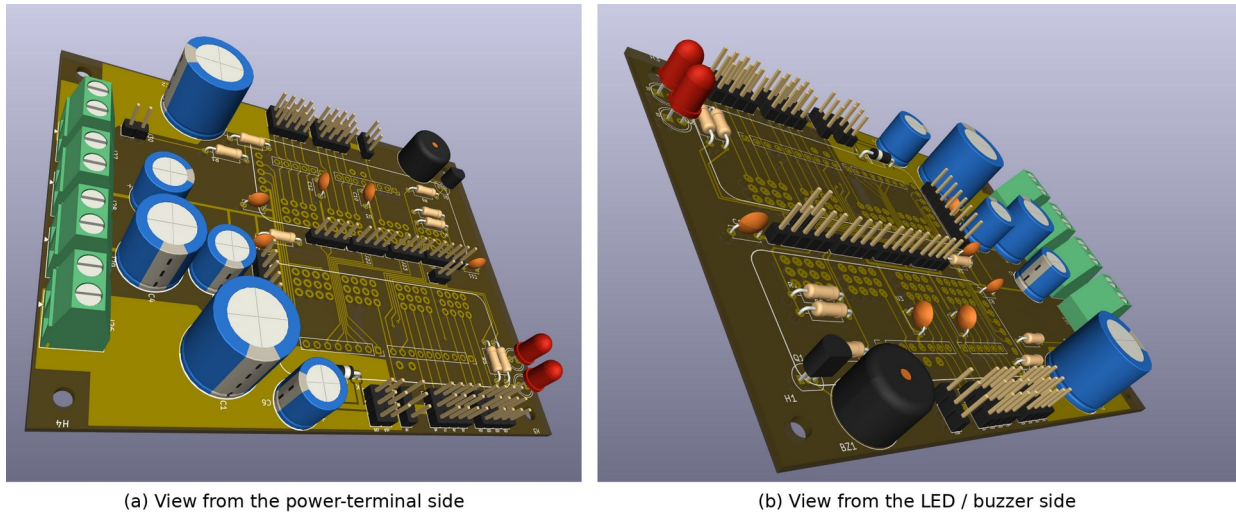


Figure 6.6: 3D views of the PCB from both sides

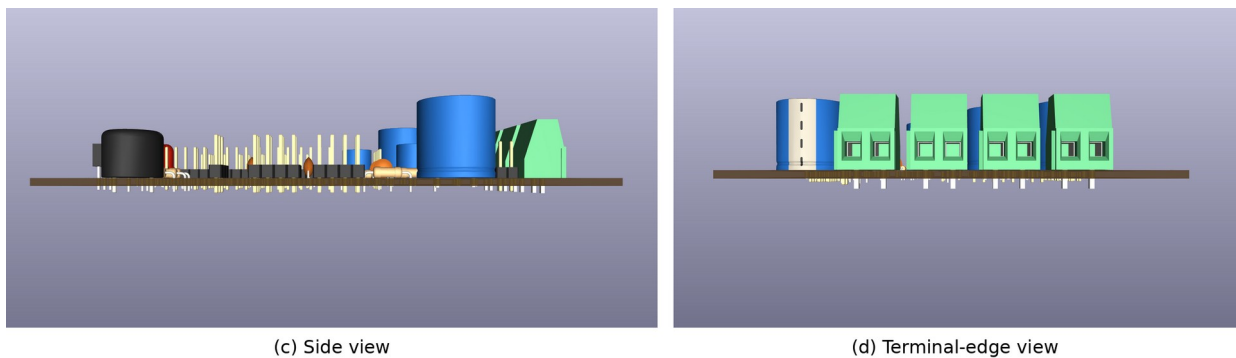


Figure 6.7: Side and edge views of the PCB, showing component heights

From the 3D views we made the following observations about the board:

- **Power terminals:** the four screw terminals J26–J29 sit in a row along one edge with their wire entries facing outwards, so the converter leads can be tightened from the side without reaching over other parts (Figure 6.7(d)).
- **Power copper areas:** the wide +6V_A and +6V_B areas are clearly visible in the top view (Figure 6.5), running from J26 and J27 to the two rows of servo sockets.
- **Servo sockets:** the three-pin servo headers are grouped in labelled blocks along the top and bottom edges, so each leg's cables leave the board in one bundle.
- **Component height:** the tallest parts are the 2200 μF capacitors C1 and C2 and the screw terminals (Figure 6.7(c)). The electronics box, which will be designed during the 3D-printing stage, must leave enough height above these parts, and also above the PCA9685 modules, which plug into their headers from above.
- **Leads under the board:** the side views show the through-hole leads coming out underneath the board. Because the shield sits directly on top of the Arduino Uno, these leads will be

trimmed short after soldering and checked for clearance from the Uno's USB and power jacks, so that nothing on the underside can short against the Arduino.

- **Indicators:** the two LEDs and the buzzer BZ1 are placed at the board edge, where they will be visible and audible through the electronics box.

The main board parameters are summarised in Table 6.5.

Table 6.5: PCB design data

Parameter	Value
Board type	Shield for the Arduino Uno R3 (stacking headers J1–J4)
Board size	About 114 × 84 mm (as per the design plan)
Number of copper layers	Two (front and back copper)
Power distribution	Copper areas for +6V_A and +6V_B; ground pour on the back layer
Mounting	Four corner mounting holes (H1–H4)
Components	Through-hole parts throughout
Fabrication	Gerber files ready; order pending

6.6 Design Verification

Before exporting fabrication files, the design was checked with KiCad's Electrical Rules Checker (ERC) on the schematic and Design Rules Checker (DRC) on the layout. The netlist was also cross-checked by hand against our connection tables, and the footprints against the real modules we had purchased.

6.6.1 Electrical Rules Check (ERC)

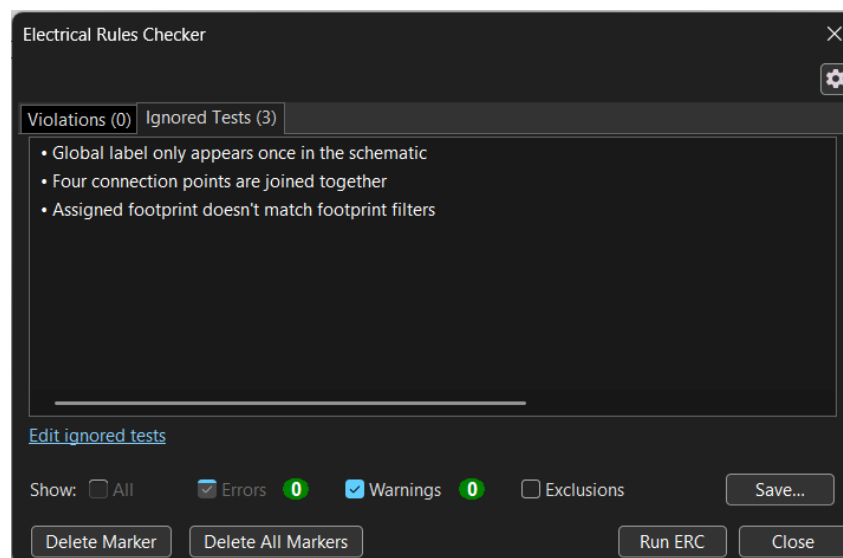


Figure 6.8: ERC result — 0 violations, 0 errors and 0 warnings

The final ERC run reports **0 violations, 0 errors and 0 warnings** (Figure 6.8). Three ERC tests were set to “ignored”. We looked at each one and they are not needed for a board like ours:

- **“Global label only appears once in the schematic.”** This warning appears when a net label is used on only one wire. For us, some labels (for example VBAT_SENSE) are used only to give a net a readable name. The pins on that net are still joined by wires, so the connection is not affected; the label is there for readability.
- **“Four connection points are joined together.”** KiCad warns about four-way junctions because a crossing and a junction can look alike on a printed schematic. In our schematic these are deliberate junctions, and each one is marked with a junction dot, so there is no doubt about the connection.
- **“Assigned footprint doesn't match footprint filters.”** This happens when a footprint is chosen that is not in the symbol's suggested list. On our board this is intentional: the PCA9685 modules, sensor headers and screw terminals use footprints we chose to match the actual purchased parts, which were checked physically.

None of these three tests concerns an electrical fault, so leaving them ignored does not hide a real error. Pins that are intentionally unused carry no-connect flags (Section 6.4), which is why ERC reports no unconnected-pin errors.

6.6.2 Design Rules Check (DRC)

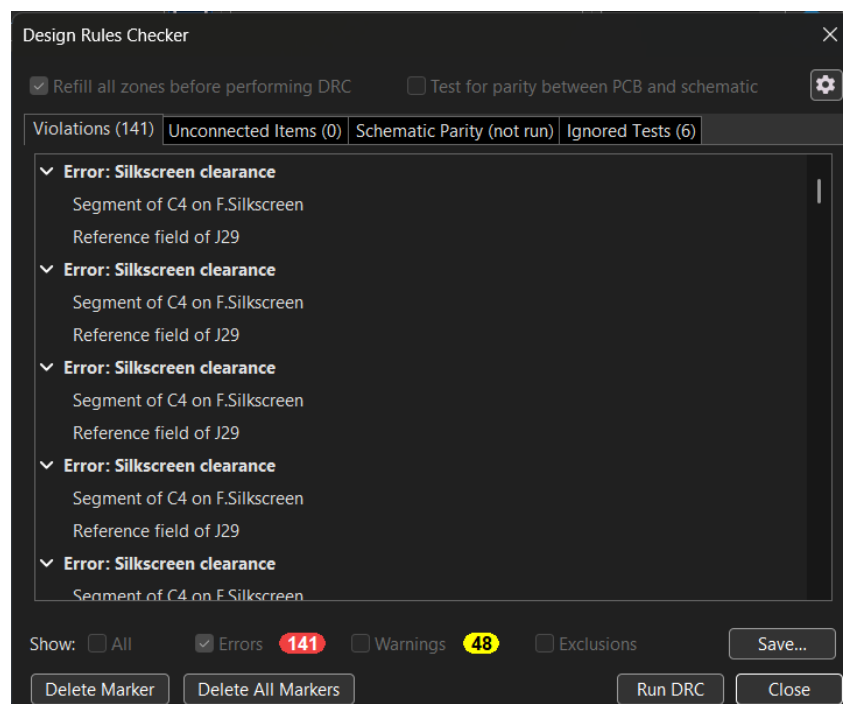


Figure 6.9: DRC result — 0 unconnected items; the listed violations are silkscreen clearance

The DRC result is shown in Figure 6.9. The most important number here is **Unconnected Items: 0** — every net in the schematic is routed on the board, and no connection is missing. The DRC also lists 141 errors and 48 warnings. The errors are of the **silkscreen clearance** type, for

example the outline of capacitor C4 on the front silkscreen layer sitting too close to the reference text of J29. These concern only the printed white markings on the board. They do not change any copper track, pad, clearance between copper or connection, so they do not affect how the board works electrically. We know the connections of this board in detail and checked them against our connection tables, so we treated these markings-related errors as acceptable for this board.

A practical note: when the board is fabricated, the manufacturer usually clips any silkscreen that overlaps a pad, so some labels may appear slightly trimmed. This is cosmetic only. If time allows before ordering, the overlapping labels can be moved to make the printed markings neater.

6.6.3 Other Checks

- **Cross-checking the netlist against our connection tables** — in particular that the V+ pads of the driver modules belong to no net, the two 6 V zones are separate, the camera 5 V node is isolated from the logic 5 V, and D0/D1 and VIN are unconnected.
- **Footprint checks against the real modules** that we had purchased, using the physical parts.

One more check is planned just before placing the order: printing the layout on paper at 1:1 scale and placing the actual modules, headers and terminals on it to confirm the footprints and spacing by eye. This is a cheap way to catch a footprint mistake before paying for fabrication.

6.7 Gerber Generation

After the checks, the fabrication outputs were exported from KiCad: Gerber files for the copper, solder-mask, silkscreen and board-outline (edge cut) layers, together with the drill file. These files are what a PCB manufacturer uses to make the board, and they are ready to be uploaded for fabrication.

The exported file set (project name “hex-pcb”, exported on 27 September 2026) is listed in Table 6.6.

Table 6.6: Fabrication files exported from KiCad

File	Contents	Size
hex-pcb-F_Cu.gbr	Front copper layer (tracks and pads)	66 KB
hex-pcb-B_Cu.gbr	Back copper layer, including the ground pour	426 KB
hex-pcb-F_Mask.gbr / B_Mask.gbr	Solder mask, front and back	12 KB each
hex-pcb-F_Silkscreen.gbr / B_Silkscreen.gbr	Silkscreen (labels and outlines), front and back	182 KB / 12 KB
hex-pcb-F_Paste.gbr / B_Paste.gbr	Solder paste layers — practically empty, as all parts are through-hole	1 KB each
hex-pcb-Edge_Cuts.gbr	Board outline	1 KB

File	Contents	Size
hex-pcb-PTH.drl	Drill file for plated through-holes (component pins, vias)	6 KB
hex-pcb-NPTH.drl	Drill file for non-plated holes (mounting holes)	1 KB
hex-pcb-job.gbrjob	Gerber job file describing the layer stack for the manufacturer	3 KB

The back copper file is by far the largest because it contains the ground pour that covers most of the back of the board. These files will be compressed into a single ZIP archive and uploaded when the board is ordered.

6.8 Current Status of the PCB

Status

The PCB design has been completed and the Gerber fabrication files have been generated. PCB fabrication/ordering is the next pending step.

The board has not yet been fabricated or received, and it has not been soldered or tested.

Table 6.7: PCB milestone status

PCB stage	Status
Connection definition	Completed
Schematic in KiCad	Completed
ERC	Completed — 0 errors, 0 warnings
Footprint assignment	Completed
PCB layout	Completed
DRC	Completed — 0 unconnected items; silkscreen-only violations
Gerber and drill export	Completed
1:1 paper print check	Planned before ordering
Ordering / fabrication	Pending — next step
Soldering and continuity checks	Planned after the board arrives
Bring-up (power on, I ² C scan)	Planned

6.9 Checks Planned After the Board Arrives

When the fabricated board arrives, it will be checked before any module is fitted:

- continuity check that no power rail is shorted to ground;
- the two 6 V zones are not connected to each other, and the camera node is not connected to the logic 5 V;
- the 6 V zones actually reach pin 2 of the correct servo sockets;
- after soldering: trim all through-hole leads on the underside and check that nothing touches the Arduino Uno's USB or power jack when the shield is plugged in;

- after fitting modules: power on without servos and confirm that nothing gets warm and an I²C scanner sketch finds 0x40, 0x41 and 0x68;
- with the Arduino held in reset, all servo outputs should be limp, which proves the OE pull-up works.

CHAPTER 7

COMPONENT PROCUREMENT

7.1 Procurement Approach

Components were selected according to the build plan and bought from Indian online suppliers. A few procurement decisions were made on purpose:

- **All MG996R servos from one seller in one order.** Different batches can have slightly different splines and centre positions, which would mean calibrating joints twice. We bought 19 (18 + 1 spare).
- **Avoiding very cheap “MG996R” servos.** Servos sold at very low prices are often fakes with plastic internal gears and much lower torque. Under a robot of about 2 kg they would fail quickly.
- **Two identical LiPo packs bought together,** same brand and capacity, so that either can be fitted.
- **Both PCA9685 boards of the same model,** since only the address jumper should differ between them.
- **Modules bought early,** because the real modules were needed on the bench to check PCB footprints.

7.2 Components Procured

Table 7.1 lists the components, their quantities as per the build plan and their status. Items marked “already available” were owned by the team before the project and are being reused. **Purchased** means the item has been bought; it does not mean it has been installed or tested — none of the components has been integrated into the robot yet.

Table 7.1: Component procurement status

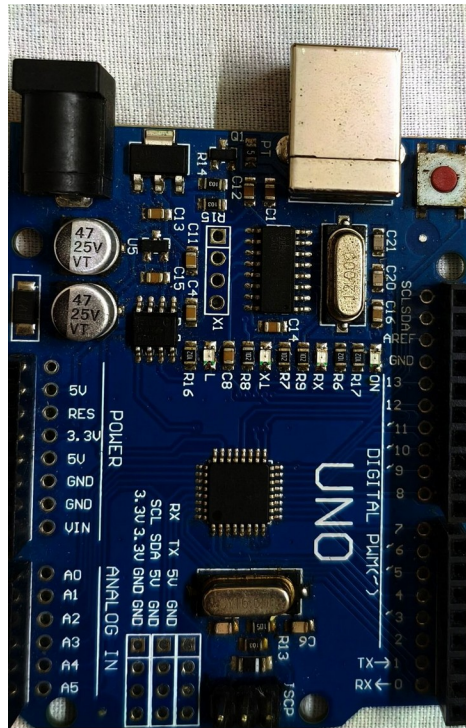
Component	Qty	Purpose	Status
Arduino Uno R3	1	Main controller	Already available (owned)
MG996R metal-gear servo	19	Leg joints (18) + 1 spare	Purchased
Aluminium servo horns (MG996R)	6	Thigh joints (carry body weight)	Purchased
Spare plastic servo horns	4	Spares for assembly	Purchased
SG90 micro servo	2	Camera pan and tilt	1 owned + 1 purchased
PCA9685 16-channel PWM driver	2	Servo control over I ² C	Purchased
ESP32-CAM (AI-Thinker, OV2640)	1	Wireless video surveillance	Purchased
HC-SR04 ultrasonic sensor	3	Obstacle sensing (front, left, right)	Purchased
MPU6050 / GY-521 IMU	1	Orientation (tilt) sensing	Purchased
XL4016 buck converter (8–10 A)	2	6 V servo rails	Purchased
LM2596 buck converter	2	5 V logic rail; 5 V camera rail	1 owned + 1 purchased

Component	Qty	Purpose	Status
3S LiPo 11.1 V 2200 mAh, XT60	2	Main battery + charged spare	Purchased
LiPo balance charger	1	Safe charging	Purchased
LiPo fireproof charging bag	1	Charging safety	Purchased
Inline blade-fuse holder + 20 A fuses	1 + 3	Battery protection	Purchased
20 A rocker switch	1	Master power switch	Purchased
XT60 connector pairs	3	Battery / supply connection	Purchased
Electrolytic capacitors 2200 μ F / 16 V (low ESR)	2	C1, C2 on servo zones	Purchased
Electrolytic capacitors 470 μ F / 16 V	3	C3, C5, C6	Purchased
Electrolytic capacitor 1000 μ F / 25 V	1	C4 on camera rail	Already available (owned)
Ceramic capacitors 100 nF	10	C7–C12 decoupling (+ spares)	Purchased
1N4007 diode	2	D1 camera-servo feed (+ spare)	Purchased
Resistors (100 k, 47 k, 10 k, 2 k, 1 k, 330 Ω), 2 LEDs, BC547, 5 V active buzzer	1 set	Support circuits on the PCB	Purchased
Female header strips, 2.54 mm, 40-pin	8	Sockets for drivers and IMU	Purchased
Male pin headers, 2.54 mm	$\approx$ 100 pins	Servo sockets and sensor headers	Already available (owned)
Arduino stacking header set (R3)	1	Shield mounting	Purchased
2-pin screw terminals, 5.08 mm, 15 A	8	Power inputs + spares	Purchased
M3 standoffs (nylon/brass), 6 / 16 mm	12	Mounting Arduino and PCB	Purchased
M3 brass heat-set inserts	100	Threads in printed parts (about 75 used)	Purchased
M3 socket-head screws (8–20 mm)	$\approx$ 100	Structural fastening	Purchased
M2 / M2.5 screws	$\approx$ 40	Horns, sensor clamps, camera mount	Purchased
M3 nyloc nuts	30	Vibration-proof fastening	Purchased
Ball bearings for hip joints	6–12	Hip joint support	Purchased
Rubber foot caps	6	Grip on smooth floors	Purchased
Silicone wire 14 / 16 / 24 AWG	as per plan	Battery, converter and signal wiring	Purchased
Dupont jumper kit, 30 cm servo extensions (6)	1 kit + 6	Sensor, camera and far-leg cables	Purchased
Heat-shrink, cable ties, spiral wrap, labels	1 set	Cable management and labelling	Purchased
12 V 1 A AC-DC adapter	1	Bench supply for early tests	Already available (owned)
ESP32-WROOM-32 dev board	1	Programmer for the ESP32-CAM	Already available (owned)

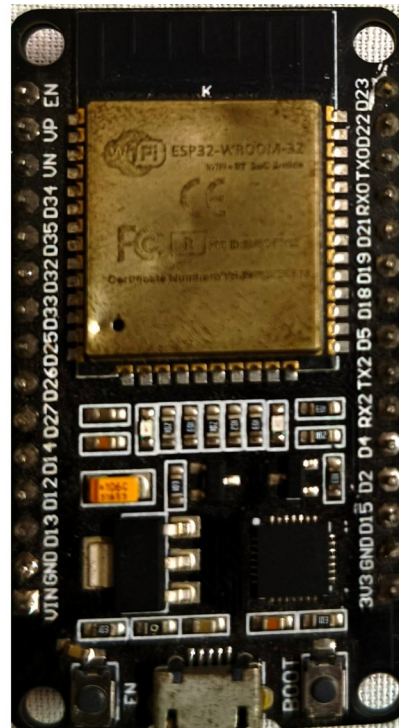
Two optional items in the build plan — a microSD card for on-board recording and a 12 V 8–10 A switching supply for tethered table-top demonstrations — are not part of the required list and are not included in the table above.

Figure 7.1 shows some of the procured components laid out together. It is not a complete set — for example, only one MG996R, one HC-SR04 and one PCA9685 are shown — but it gives an idea of the main parts: an XL4016 and an LM2596 buck converter, the Arduino Uno R3, the ESP32-WROOM-32 development board, a PCA9685 driver, the 3S 2200 mAh LiPo pack with its XT60 connector, the B3 balance charger, an HC-SR04 sensor, an MG996R and an SG90 servo, the resistor kit, capacitors, 1N4007 diodes and pin headers. None of these parts has been installed on the robot yet.





(a) Arduino Uno R3

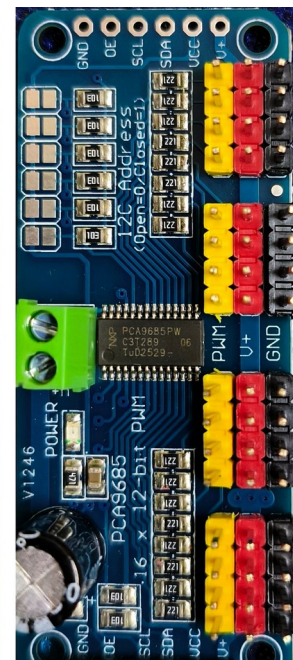


(b) ESP32-WROOM-32 dev board

Figure 7.2: (a) Arduino Uno R3 — main controller; (b) ESP32-WROOM-32 development board — programmer for the ESP32-CAM



(c) MG996R servo (TowerPro)



(d) PCA9685

Figure 7.3: (c) TowerPro MG996R metal-gear servo — leg actuator; (d) PCA9685 16-channel PWM driver

7.3 Components Deliberately Not Used

A few parts we already owned were considered and left out. An **L298N motor driver** drives DC motors, and every moving part of this robot is a servo, so there is nothing for it to drive; it would

also add about 30 g. A **4-channel relay module** was considered for cutting servo power in an emergency, but the OE line of the driver boards already does this electronically in microseconds, and relays switching several amperes of DC tend to arc. The relay module will instead be used on the test bench as a switch for the bench supply.

7.4 Estimated Cost

The build plan estimates the cost of the purchase list, based on typical Indian online prices checked in September 2026. These are planning estimates; actual prices paid vary by seller.

Table 7.2: Estimated cost of purchased components (from the build plan)

Group	Estimated subtotal (₹)
A — Legs and motion (19 servos, horns, SG90)	7,681
B — Servo drivers	600
C — Camera (ESP32-CAM)	550
D — Sensing	340
E — Power system (two battery packs)	5,115
F — Parts that go on the PCB	700
G — Mechanical hardware	1,310
H — Wire and connectors	1,050
Total estimated purchase cost	17,346
Value of parts reused from team stock (not purchased)	≈ 1,205
PCB fabrication	Not included
3D printing and filament	Provided by the college

Our earlier presentation mentioned a target of under ₹12,000. The detailed build plan came out higher, mainly because of 19 genuine MG996R servos and two battery packs, which we decided not to compromise on.

CHAPTER 8

MECHANICAL DESIGN AND 3D PRINTING PLAN

8.1 Current Stage

With the components purchased and the PCB ready for fabrication, **the next stage is to begin 3D printing and mechanical prototyping**. The college provides the 3D printing facility and material.

Status of the mechanical stage

We have not started the 3D printing phase yet. For this reason, this report does not contain any photographs of printed parts or of an assembled robot. This chapter describes what will be printed, why dimensional accuracy matters, and the order in which printing will be done. Photographs of the printed parts will be added in the final report.

8.2 Parts to Be Printed

Table 8.1: Planned 3D-printed parts

Part	Qty	Purpose / design notes
Leg assemblies (coxa, femur, tibia segments and servo brackets)	6 sets	Coxa 30 mm, femur 65 mm, tibia 90 mm; about 57 g per leg planned; hip joint with bearing
Body plate	1	4 mm thick with a stiffening rim; carries the six hip servos
Electronics box + lid	1	2.4 mm walls; holds Arduino with shield, sensors; ultrasonic sensors in the front and side walls; switch in the rear wall
Camera pan-tilt mount	1	Thin-walled, over the centre of the box; holds ESP32-CAM and two SG90s
Battery tray	1	Under the body plate; holds the LiPo pack, easy to swap
Converter tray	1	Open, slotted tray under the body plate for air cooling of the four converters
Arduino sled / mounting parts	as needed	Mounting the Uno and shield inside the box on standoffs
Sensor clamps	3	Holding the HC-SR04 sensors in the box walls
Test pieces	—	Tolerance cube and one servo bracket before the full leg
Spare tibias	2	Planned spares for the demonstration

We plan to start from an existing leg model, study it, and adapt it to our link lengths and to the dimensions of our actual servos, rather than design every leg part from zero.

[The 3D printing phase has not been started yet, so no image is available at this stage. The 3D model of the hexapod will be added here once it is ready.]

Figure 8.1: 3D model of the hexapod

8.3 Why Dimensional Accuracy Matters

For a walking robot, small mechanical errors turn directly into walking problems, and most of them cannot be corrected in software:

- **Servo mounting dimensions:** the servo pocket must hold the MG996R body firmly. If it is loose, the servo body rocks inside the bracket and the joint has play; if it is tight, the part cracks or the servo cannot be inserted. Printed holes and pockets usually come out slightly smaller than modelled, which is why a tolerance test comes first.
- **Screw holes and inserts:** screws threaded directly into printed plastic strip after a few removals, faster under servo torque. We will use M3 brass heat-set inserts in every screw hole in printed parts, which needs the holes to be the correct size for the insert.
- **Clearances:** each leg must move through its full range without rubbing against the body or the next segment. A collision means the servo stalls and draws about 2.5 A.
- **Alignment:** the hip, thigh and knee axes must be square to each other. A twisted bracket means the foot does not land where the inverse kinematics expects it to.
- **Weight:** the leg count multiplies every gram by six. The total must stay under about 2.3 kg to keep the thigh servos below 60 % of stall torque.
- **Strength:** the thigh brackets carry the body weight and the shock of each foot landing, so infill and wall thickness have to be chosen for strength, not just appearance.

8.4 Printing and Prototyping Workflow

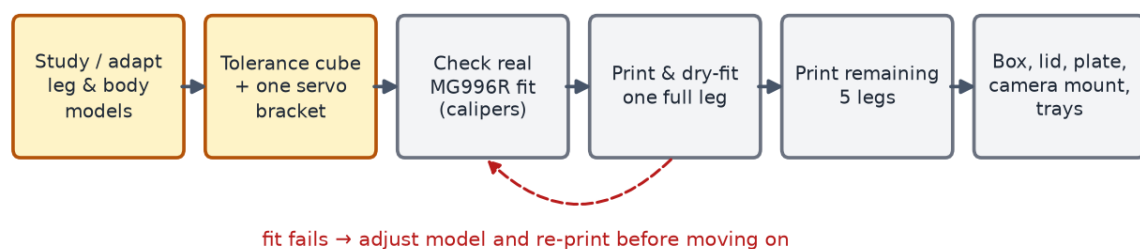


Figure 8.2: Planned 3D printing workflow — each step is checked before the next

1. Book printer time in the college lab, since slot booking has its own queue.
2. Print a tolerance cube and one servo bracket, and check that a real MG996R fits the printed bracket.

3. Print and dry-fit one complete leg: three printed segments, three servos, one bearing and all heat-set inserts. Move it through its full range by hand; it should not bind or rub anywhere.
4. Only then print the remaining five legs (the build plan estimates about 20 hours of printer time for these).
5. Print the electronics box, lid, camera mount, body plate and both trays, and check that the real PCB and Uno fit the printed box.
6. Print foot-cap fittings and spare parts.

The build plan estimates about 25 hours of printer time for the legs, body and box together. While the parts are printing, the PCB order and the bench power tests can proceed in parallel.

8.5 Mass Placement for Stability

The mechanical layout follows directly from the stability discussion in Chapter 3. The battery and converters (about 245 g) are placed under the body plate, at the lowest point of the robot, which offsets the camera turret on top. The turret is kept light and over the centre of the box, not out on the nose, because a heavy camera on a long arm would raise the centre of mass and push its projection towards the edge of the support triangle during turns. The planned standing height of the body is 70–90 mm off the floor; standing taller looks impressive but uses more battery and makes tipping more likely.



Figure 8.3: First 3D-printed test parts

CHAPTER 9

CURRENT PROJECT PROGRESS

9.1 Progress Summary

The project has moved beyond the planning stage. The table below gives the status of each stage. We have not given percentage figures, because the stages are very different in size and a percentage would not be meaningful; the table shows which stages are actually complete.

Table 9.1: Current project progress

Project stage	Status
Project concept	Completed
Literature study and base-paper review	Completed
System architecture	Completed
Component selection	Completed
Component procurement	Completed
PCB schematic / design	Completed
PCB layout	Completed
PCB design verification (ERC, DRC, cross-checks)	Completed
Gerber generation	Completed
PCB fabrication / order	Pending — next step
3D mechanical design / printing	Not started — next stage
Mechanical assembly	Pending
Electronics integration	Pending
Firmware / gait programming	Pending
Sensor integration	Pending
Camera integration	Pending
Testing	Pending
Final demonstration	Pending

9.2 Work Completed So Far

1. **Finalised the architecture:** Arduino Uno R3 as controller, two PCA9685 drivers, 18 MG996R leg servos, pan-tilt ESP32-CAM, three HC-SR04 sensors, MPU6050, and a four-converter power system.
2. **Completed the design calculations** that decide the hardware: current budget per rail, converter ratings, fuse and wire sizes, weight budget and thigh-servo torque.
3. **Selected the components**, including the decisions on what not to use.
4. **Purchased the components** listed in Chapter 7.
5. **Designed the custom PCB** in KiCad as an Arduino Uno shield with separate power zones, 20 labelled servo sockets, sensor headers and an isolated camera supply.
6. **Completed the PCB layout** and ran the design checks.

7. **Generated the Gerber and drill files**, which are ready for fabrication.

9.3 Work in Progress

- Placing the PCB order (the only remaining step for the PCB itself).
- Preparing for the 3D printing stage, which has not started yet: booking printer time in the college lab and choosing the leg model to adapt.

9.4 Work Remaining

- PCB fabrication, soldering and continuity checks.
- Setting and verifying the four converters; building the power harness.
- 3D printing of all structural parts and dry-fitting.
- Mechanical assembly of the legs, body, box and camera turret.
- Electronics integration: fitting the shield, modules, sensors and cables.
- Firmware: servo calibration, inverse kinematics, tripod gait, IMU reading, ultrasonic checks, camera web page and serial command set.
- Sensor and camera integration and calibration.
- Electrical, mechanical, thermal and behavioural testing; battery-life measurement; demonstration rehearsal.

CHAPTER 10

CHALLENGES AND ENGINEERING CONSIDERATIONS

Most of the design decisions so far were made to avoid problems that commonly end projects like this one. The main ones are listed below, together with how our design deals with them. Several of them can only be confirmed once the robot is built.

Table 10.1: Key engineering challenges and how the design addresses them

Challenge	Risk if ignored	How we address it
High servo current	About 11 A while walking; a small converter overheats or shuts down mid-walk	Two XL4016 converters (8–10 A), one per side; wide copper zones on the PCB
Camera current bursts	Wi-Fi bursts pull down the 5 V rail and reset the Arduino	Dedicated converter D and bulk capacitors at the camera
Ground loops	Random I ² C failures only when the robot moves	Single star ground point; one ground pour on the PCB
Power-on twitch	All servos jump to random positions at start-up	OE pull-up keeps outputs off until a safe pose is written
Mis-set converter	11 V on the 6 V rail destroys all servos	Set and measure each converter on the bench before connecting servos
Weight	Thigh servos near stall; overheating and wear	Weight budget before printing; battery and converters low; weigh while building
Printed threads stripping	Joints develop play; gait cannot be tuned	Brass heat-set inserts in every printed hole
Servo centring	Joint zero off by 10–30°	Command each servo to centre before fitting the horn
Cable wind-up on camera	Connector pulled out while panning	Pan limited to $\pm 80^\circ$ in firmware; service loop in the cable
Limited RAM (2 KB)	Crashes or unstable behaviour	Gait tables in PROGMEM; no String class; integer maths
LiPo safety	Fire risk during charging or over-discharge	Balance charger, fireproof bag, fuse, low-battery cut-off at 9.9 V
Heat in converters	Thermal shutdown	Open, slotted converter tray in free air

10.1 Challenges Specific to the Current Stage

- **PCB lead time:** once ordered, the board takes time to arrive. The plan keeps printing, converter setting and bench tests running in parallel so that the board does not block everything.
- **Printer availability:** the college printers are shared, so print jobs have to be planned and booked in advance.
- **Print tolerances:** the first printed brackets may not fit the servos exactly. The workflow in Chapter 8 includes test prints for this reason.
- **Unverified estimates:** current, weight, torque and runtime figures are design estimates. We plan to measure real current draw during single-leg tests and weigh the robot during

assembly. If the measured current is much higher than expected, the build plan's fallback is to add a third XL4016 and split the legs into three groups of six.

CHAPTER 11

EXPECTED WORKING OF THE ROBOT

This chapter describes how the robot is expected to work once it is built and programmed. It describes the intended behaviour, not tested behaviour.

11.1 Start-Up Sequence

1. The operator switches on the robot at the rocker switch in the rear wall.
2. The Arduino boots with D10 held high by the pull-up, so all servo outputs are disabled and nothing moves.
3. The firmware initialises both PCA9685 boards at 50 Hz, writes a safe pose to all 20 channels, and only then pulls D10 low to enable the servos.
4. The ESP32-CAM connects to Wi-Fi and serves its web page; the operator opens it on a phone to see the live video and the control buttons.

11.2 Normal Operation

- **Walking:** the operator sends forward, back, left, right or stop commands from the web page. The camera passes each as a single character (F, B, L, R, S) to the Arduino, which runs the tripod gait: three legs support the body while the other three swing forward, and then they swap.
- **Surveillance:** the camera streams video to the phone. The operator can pan and tilt it using P and T commands followed by a number; pan is limited to $\pm 80^\circ$ and tilt to about -30° to $+60^\circ$, or whatever the printed mount allows.
- **Obstacle avoidance:** the three ultrasonic sensors are read in the main loop. If an obstacle is detected closer than the set distance (30 cm planned) in the direction of motion, forward motion is stopped regardless of the operator's command.
- **Balance:** the MPU6050 reports body pitch and roll. The firmware can use this to correct the body posture on slopes, and, as discussed in Chapter 3, to decide when a gait with more legs on the ground is needed.
- **Battery monitoring:** the battery voltage is read on A0. Below 10.5 V the robot warns the user (red LED and buzzer); at 9.9 V it sits down and disables the servos to protect the battery.

11.3 Battery Swap and Tethered Operation

For a longer demonstration, the operator sits the robot down, switches off, swaps the battery for the charged spare and switches on again; because a swap is just a power cycle, the start-up sequence above keeps it safe. For table-top reviews, the build plan also allows the robot to run from a 12 V mains supply of sufficient rating through the same XT60 connector, which physically prevents a battery and the supply from being connected at the same time.

CHAPTER 12

FUTURE PLAN OF ACTION

12.1 Tentative Schedule

The schedule below is tentative. It depends on PCB delivery time and printer availability, and will be updated in the final report.

Table 12.1: Tentative plan of action

Period	Electronics	Mechanical	Checkpoint
Up to Sep 2026	Architecture, component purchase, PCB design, layout, checks and Gerber files	Mechanical plan, weight and torque budget	Completed
Oct 2026	Order PCB; set and verify all four converters on the 12 V bench adapter; build power harness; single-servo test	Book printer; tolerance cube and servo bracket; print and dry-fit one complete leg	Converters correct under load; one leg moves through its full range
Nov 2026	Receive PCB; solder, continuity check, fit modules, I ² C scan; program ESP32-CAM and test video on a phone	Print remaining five legs, box, lid, body plate, camera mount and trays	Board finds 0x40, 0x41, 0x68; parts fit the real PCB and Uno
Dec 2026	Full electronics integration; servo calibration; inverse kinematics and tripod gait	Full assembly; centre servos before fitting horns; fit camera turret	All 18 leg servos reach commanded angles with feet off the ground
Jan 2027	Sensor integration, obstacle interlock, IMU correction, battery-life measurement	Foot caps, cable management, spare parts	Robot stands, walks and streams video; demonstration

12.2 Planned Assembly and Test Order

The assembly will follow a fixed order so that each expensive part is proven before it meets anything else. Steps 1–6 will be done on our 12 V 1 A bench adapter instead of the LiPo, so that a wiring mistake trips the adapter instead of sending a large current through the board.

1. Set all four converters on the bench (A and B to 6.00 V, C and D to 5.00 V, within 0.05 V, with and without load).
2. Build and test the power harness alone — fuse, switch, terminal strips, converters; no PCB.
3. Continuity-check the bare PCB.
4. Power the board without servos; fit drivers and IMU; I²C scan.
5. Test a single servo on socket J31 (R1-C) with a sweep sketch, and measure its real current draw.
6. Program the ESP32-CAM and confirm live video on a phone and a test character arriving at the Arduino.
7. Print and dry-fit one leg.
8. Centre every servo before fitting its horn.

9. Print the remaining legs, body and box.
10. Fit the box, Arduino, board and sensors; converters and battery into their trays.
11. Attach the legs one at a time, checking each leg's three channels after fitting.
12. Fit the camera turret last.
13. First full power-on with feet off the ground (battery or a large enough supply from here).
14. First stand: hold its own weight for 60 seconds without sagging or servo chatter.
15. First steps: one leg cycle at a time, then a slow tripod gait, then speed.

12.3 Testing Plan

Testing will use the checklists from our build plan, grouped into four areas:

- **Electrical:** converter outputs under load, about 5.2 V at the camera servo pins, fuse and switch in place, no rail shorted to ground, the two 6 V zones and the two 5 V nodes separate, battery sensing within 0.2 V of a multimeter, I²C scan result, servos limp with the Arduino held in reset.
- **Mechanical:** inserts in every printed screw hole, bearings in all hip joints, full range of every leg without binding, horns fitted at centre, cable service loop on the camera, total weight under 2.3 kg, rubber foot caps.
- **Thermal (after five minutes of movement):** converters warm but not too hot to touch, no servo much hotter than the others, camera and battery within normal temperature.
- **Behavioural:** every servo on the correct channel, left and right not swapped, camera pan and tilt in the right directions, video visible from at least 10 m, web commands reaching the Arduino, ultrasonic readings sensible at 20, 50 and 100 cm, IMU reading about zero on a level table, low-battery warning at the right voltage.

The results of these tests will be reported in the final project report.

CHAPTER 13

EXPECTED FINAL OUTCOME

If the remaining stages go as planned, the expected outcome of the project is:

- A working six-legged robot, about 2.2 kg, with 18 servo-driven joints, controlled by an Arduino Uno R3 through a custom PCB shield.
- Walking on flat and moderately uneven ground using a tripod gait, with a slower four-legs-down gait available for rough ground.
- Live Wi-Fi video on a phone from a pan-tilt camera, with the robot controlled from the same web page.
- Automatic stopping in front of obstacles using the three ultrasonic sensors, and tilt sensing through the MPU6050.
- A safe power system with separate rails, a fuse, battery monitoring and a controlled start-up.
- Documented design files — schematic, PCB, Gerber files, printed-part models and firmware — that another student team could reproduce.

The final report will add the measured results: actual weight, current draw, converter temperatures, battery runtime, walking behaviour, sensor accuracy and video range. We will report these as measured, including anything that does not meet the design estimates.

CHAPTER 14

CONCLUSION

This report has described the progress of our final-year project, the Arduino-Based Hexapod for Emergency and Surveillance, up to the end of the design and procurement stage.

So far, we have finalised the project concept and the system architecture, studied the relevant literature and our base paper, and selected and purchased the required components. The most significant completed milestone is the custom PCB: it has been designed in KiCad as a shield for the Arduino Uno with separate power zones, labelled servo sockets, sensor headers and an isolated camera supply; its layout has been completed and checked; and the Gerber fabrication files have been generated. PCB fabrication is the next pending step.

From our base paper we have taken the idea of the support polygon and the static stability margin, and used it to guide our choice of gait and the placement of mass in the robot. We have not implemented the paper's reinforcement-learning controller, which is beyond the capability of our controller.

The next stage is mechanical fabrication through 3D printing, which has not started yet. The remaining work involves getting the PCB fabricated, printing and assembling the mechanical parts, integrating the electronics, programming the gait and control, integrating the sensors and the camera, calibration and testing. The robot has not yet been assembled, and no walking, surveillance or power results are claimed at this stage; these will be measured and reported in the final project report.

REFERENCES

- [1] S. Ji, W. Wei, X. Liu, and J. Wu, “Optimization control of attitude stability for hexapod robots based on reinforcement learning,” *IEEE Access*, vol. 12, pp. 154120–154132, 2024, doi: 10.1109/ACCESS.2024.3485506.
- [2] S. Mohamed, H. Q. Le, Y. Kim, and B. Shin, “Design and modeling of hexapod robot using telescopic legs connected to pivot joints at the hips,” *Proc. Inst. Mech. Eng., Part C: J. Mech. Eng. Sci.*, vol. 238, 2024, doi: 10.1177/09544062231198383.
- [3] C. Liu, Y. Zhu, Z. Han, *et al.*, “Design and control of a hexapod robot RENS H3 for lateral walking on unknown rugged terrains,” *J. Field Robot.*, 2025, doi: 10.1002/rob.22591.
- [4] K. Xu, R. Qin, C. Chen, G. Dong, J. Chen, and X. Ding, “Design and multimodal locomotion plan of a hexapod robot with improved knee joints,” *J. Field Robot.*, vol. 41, 2024, doi: 10.1002/rob.22324.
- [5] C. Saengsint *et al.*, “Autonomous rescue hexapod robot with AI human detection and tracking,” in *Proc. IEEE Int. Conf. Robotics and Biomimetics (ROBIO)*, Koh Samui, Thailand, 2023.
- [6] S. V. Kulkarni, N. Samanvita, S. Gatade, and R. Likhitha, “Obstacle avoidance robot based on Arduino for live video transmission and surveillance,” in *Emerging Research in Computing, Information, Communication and Applications (ERCICA 2023)*, Lecture Notes in Electrical Engineering, vol. 1104. Singapore: Springer, 2024, doi: 10.1007/978-981-99-7622-5_10.
- [7] S. Mahmud, A. A. Fime, and J.-H. Kim, “ATR HarmoniSAR: A system for enhancing victim detection in robot-assisted disaster scenarios,” *IEEE Access*, vol. 12, 2024.
- [8] C. Cruz Ulloa, D. Orbea, J. del Cerro, and A. Barrientos, “Thermal, multispectral, and RGB vision systems analysis for victim detection in SAR robotics,” *Applied Sciences*, vol. 14, no. 2, p. 766, 2024, doi: 10.3390/app14020766.
- [9] R. Jadeja, T. Trivedi, and J. Surve, “Survivor detection approach for post earthquake search and rescue missions based on deep learning inspired algorithms,” *Scientific Reports*, vol. 14, 2024, doi: 10.1038/s41598-024-75156-z.
- [10] S. Gelfert, “Real time victim detection in smoky environments with mobile robot and multi-sensor unit using deep learning,” in *Robot Intelligence Technology and Applications 7 (RiTA 2022)*, Lecture Notes in Networks and Systems. Cham: Springer, 2023, doi: 10.1007/978-3-031-26889-2_32.
- [11] L. H. Ting, R. Blickhan, and R. J. Full, “Dynamic and static stability in hexapedal runners,” *J. Exp. Biol.*, vol. 197, no. 1, pp. 251–269, 1994. (Cited in [1] as the source of the static stability margin.)
- [12] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. Int. Conf. Machine Learning (ICML)*, PMLR, 2018. (Cited in [1] as the training algorithm.)
- [13] Group B9, “Arduino-based hexapod spider robot with rotatable surveillance camera — Components, costing and complete build plan,” Document 1 of 3, Version 3, Dept. of ECE, GCET, Greater Noida, Sep. 2026 (internal project document).

APPENDIX A: PLANNED HARDWARE REFERENCE FOR FIRMWARE

This appendix collects the hardware facts that the firmware will need. It reflects the PCB design; it will be re-checked against the assembled robot.

Table A.1: Planned I²C devices

Address	Device
0x40	PCA9685 #1 (U2) — right legs R1, R2, R3 on ch 0–8; camera pan ch 9; tilt ch 10
0x41	PCA9685 #2 (U3) — left legs L1, L2, L3 on ch 0–8
0x68	MPU6050 IMU

Table A.2: Planned servo channel map

Leg	Hip (coxa)	Thigh (femur)	Knee (tibia)
R1 right front	0x40 ch 0	0x40 ch 1	0x40 ch 2
R2 right middle	0x40 ch 3	0x40 ch 4	0x40 ch 5
R3 right rear	0x40 ch 6	0x40 ch 7	0x40 ch 8
L1 left front	0x41 ch 0	0x41 ch 1	0x41 ch 2
L2 left middle	0x41 ch 3	0x41 ch 4	0x41 ch 5
L3 left rear	0x41 ch 6	0x41 ch 7	0x41 ch 8
Camera	pan: 0x40 ch 9	tilt: 0x40 ch 10	—

Table A.3: Planned firmware limits

Parameter	Planned value
PWM frequency	50 Hz on both PCA9685 boards
Servo pulse range	≈ 0.5–2.5 ms (counts ≈ 102–512, centre ≈ 307); calibrate each servo
Camera pan / tilt limits	±80° pan; –30° to +60° tilt (or as the printed mount allows)
Foot reach	Foot within 70 mm horizontally of the thigh joint
Low battery	Warn at 10.5 V; sit down and disable servos at 9.9 V
Camera command protocol	Single characters at 9600 baud: F, B, L, R, S; P/T + number